M363 Unit 13
UNDERGRADUATE COMPUTING

Software engineering with objects

Process quality management, human resources, quality assurance

Unit 13

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1619 1

1.1

CONTENTS

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Adviser, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Copy Editor, Ambertext

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

This unit explores how the software development process can be made predictable in terms of costs, timescale and quality of product. The aims are to:

- explain why the technical processes for developing software covered so far are not sufficient to ensure successful software development;
- give you an understanding of the principles underlying the management of software development, showing how this breaks down into the management of resources, the management of quality and the management of configuration;
- show you how resources are managed by means of project organisation, work-content and cost estimation, work scheduling and progress control;
- show you how software quality can be assured by marshalling the product quality measures in *Unit 11* and by drawing upon external standards such as ISO 9000-3 and CMMI;
- show you how a software product's configuration can be managed.

2 Overview of process quality

Developing software is never easy. When the software is large and complex, the process is even more difficult. We want to produce a high-quality product, but at a predetermined cost and within a predetermined timescale.

In *Units 5–7*, we saw the technical processes necessary for developing software using UML for undertaking activities associated with analysis and design.

In *Unit 11*, we saw how to assess the quality of a software product during its development and on its completion.

In this unit, we focus on how we should organise our software development team and its programme of work so that we do end up with a high-quality product developed within budget and schedule.

Fred Brooks managed the development of the major IBM operating systems during the 1960s, involving hundreds of software designers and programmers. In *The Mythical Man-Month* (Brooks, 1975), a number of software development issues are discussed from the point of view of a battle-scarred project manager. This book is regarded by many as a classic and essential reading for any would-be project manager. The primary thesis of the book is that persons and months are not interchangeable, hence the title of the book: if one person takes nine months to complete a task, it does not follow that nine people will take one month to complete the same task; nine people are likely to take much longer than a single month. A much quoted saying from the book is, 'Adding people to a late project just makes it later'. Involving more people in a software project may cause more problems than it solves. This is because of the added burden of training and intercommunication. Even if new developers are experienced, they must still get to know the new problem they are working on and learn to work within the project's organisation. Adding an extra person to a team increases the number of communication paths, each of which has to be managed. For example, there is one communication path between two people, there are three communication paths between three people, there are six between four people, ten between five people, and so on. When first published in 1975, this book gave voice to the growing realisation in the software industry of just how complex the software development process is. Developing software is not easy, it needs good management; and that management is not easy either.

This unit looks mainly at three aspects of that management: *project management*, *quality management and configuration management*. These aspects are introduced in this section with further discussion in Sections 3, 4 and 5.

2.1 Project and people management

Project management

Much of this unit focuses on *project management* in various forms. You can probably guess most of the important jobs a project manager must take on, such as:

- planning the project schedule;
- finding the right people to work on the project and assigning tasks to them;

- making sure the team is properly trained and has the proper tools and work environment;
- keeping the project on schedule and taking action if it slips;
- liaising with the customer (who might be external or another part of the same organisation as the project team);
- analysing and managing the risks;
- making sure that the lessons learned on other projects in the organisation feed into this project and that this project's lessons are passed on to others.

To this we might add the more people-focused responsibility of helping people develop their careers, and the responsibility of defending the project's resources, especially its human resources (for example, by ensuring that other projects do not make excessive use of any of your project's team members and thus delay their work on your project).

When accepting a project to develop software, it is important to have a good idea at the outset of how much effort will be required, and for how long. Work-content *estimation* is so important that we shall cover this in detail in Section 3, with the intention of enabling you to make reasonable work-content estimates for yourself.

Having made estimates, the work required must be assigned to people or teams, and to periods of time, to give a *work plan* for the project. Progress of the project is then monitored and assessed relative to this plan. These processes, of breaking down the work and scheduling it to produce a plan, and of tracking the progress of the project relative to the plan, are also discussed in detail in Section 3.

People management

The importance of people in the development of software cannot be over emphasised. After all, when things go wrong during the development of software, it is most likely because some person has made a mistake, perhaps with mitigating circumstances. One way to rectify this is to improve the *people management* of the project.

However, people management cannot be looked at simply on a project-by-project basis. Once a person has finished one project, they move on to the next, maybe spending time in between developing their own skills or in helping management processes (such as preparing proposals in response to invitations to tender). So people management has to be organisation-wide.

An important part of people management is a combination of *management* and *leadership*. Leaders maintain sight of the goal, motivate and inspire people, even though they may not be technically competent to do the tasks themselves. Managers usually have enough technical knowledge to appreciate how the subtasks fit together and are capable of coordinating them. Management can usually be learnt, but teaching leadership is much trickier as leaders tend to have some kind of personal commitment to the task. Leadership does not always need to come from the same person, so it is possible to appoint people in a hierarchy according to apparently good management skills and let leadership come from wherever it may. This is one of the motivations behind the *matrix organisation* method of people management.

Matrix organisation is a popular way of organising a software development company, or division within a larger company, to accommodate such organisation-wide people management; Figure 1 illustrates this. People (human resources) are classified according to their *function* (that is, their area of expertise, such as human–computer interaction, databases, distributed computing), through which they find their career advancement. As new projects get started by the business manager, functions are

assigned to projects, as shown by the small circles on the intersections of the project and function lines. (On any particular project, not all functions would necessarily be present.) Each circle represents one or more people.

Note that, in Figure 1, the management of software assets (or reusable components) and the management of quality assurance have been placed outside the organisational structure of projects, since these 'staff' functions transcend any particular project.

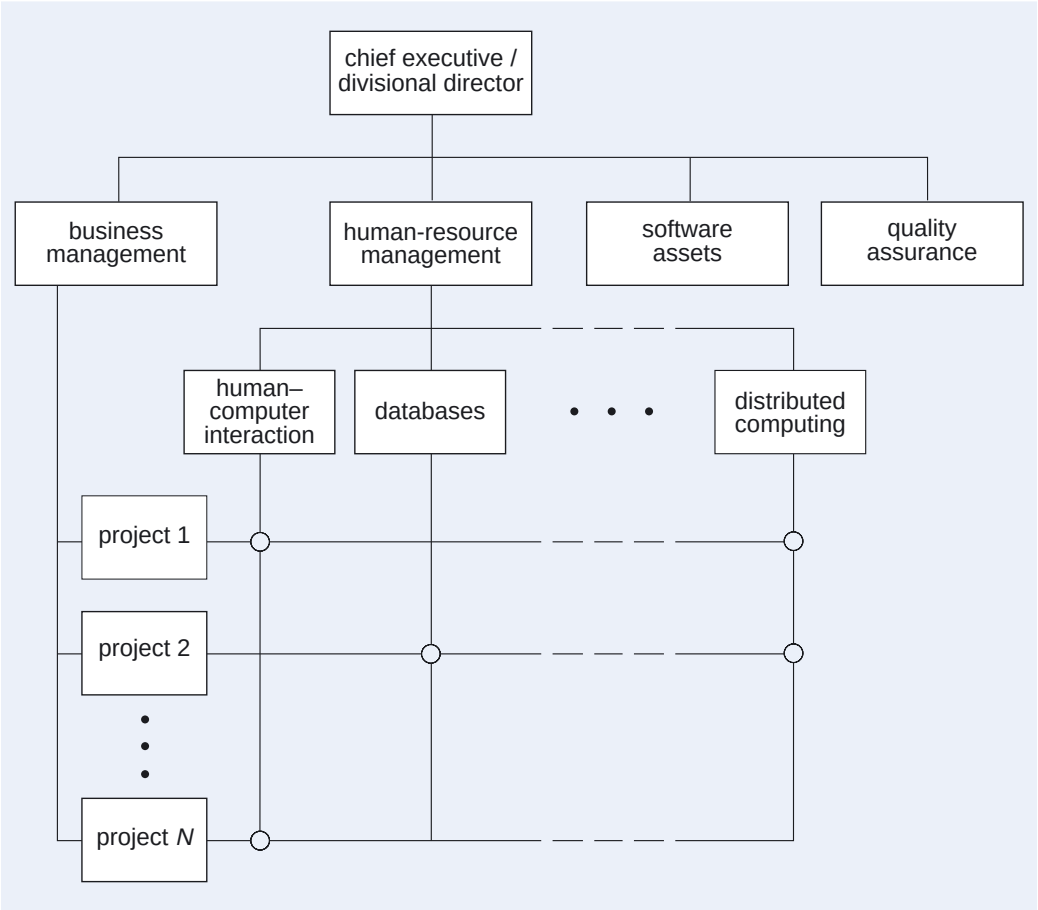

Figure 1 A matrix organisation

The matrix method of organising a company allows people to be managed appropriately at an organisational level, but they also need to be managed appropriately at project level. Each project manager must decide how to organise his/her project team so that the team members can most effectively collaborate to develop the software. We consider two common examples of *team organisation*.

The first example is separation by *task*. Here the team is divided up into subteams, with each subteam responsible for a different software development activity, as shown in Figure 2. The major advantage is that, since each subteam is responsible for a specific software development activity, there will be a consistency to the methods and tools used by each subteam. There are two major disadvantages, however. One is that this team organisation is based on the waterfall model of software development, which is now used less and less. The other disadvantage is that each subteam may develop its own culture and may lose sight of the fact that they are part of a larger project team; to try to avoid this, the project manager needs to set up clear lines of communication between the various subteams.

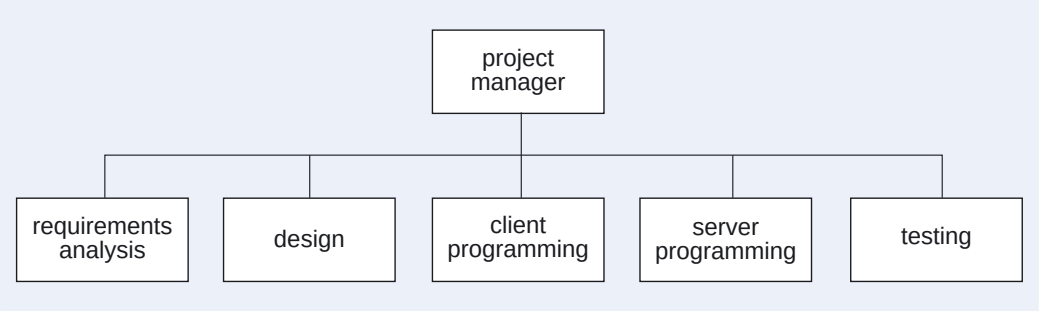

Figure 2 Task-oriented team organisation

The second example of team organisation is to arrange the subteams by *subsystem*, as Figure 3 illustrates (this assumes that the breakdown into subsystems was done during analysis). Each subteam undertakes the full set of software development activities, and has to negotiate with other subteams about interfaces between subsystems. This team organisation supports parallel development, as each subteam can work independently, though of course the subteams must coordinate their work. Communication problems are alleviated, as each subteam will be significantly smaller than the project team. However, there is a danger that each subteam will develop its own ways of working, which can endanger the integrity of the overall project.

Developers of subsystems often must have particular skill and knowledge, which enable them to be related to the functional divisions of a matrix organisation.

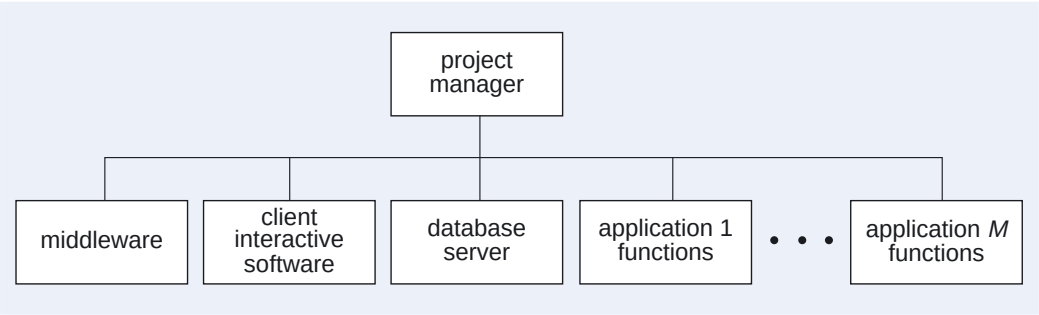

Figure 3 Subsystem-oriented team organisation

SAQ 1

If you were managing a team of ten people on a late running project and your manager offered you five more people to help get back on track, would you accept? If not, how would you justify your refusal?

ANSWER.....

Accepting may be unwise as it would create sixty more communication paths which might, in turn, make the project even later. However, the additional resources might be able to contribute effectively to bring the project back on schedule, so a definitive answer to this question is not possible.

$60 = (15 \times 14)/2 - (10 \times 9)/2$

If there are 10 people, each of them may communicate with any of the 9 others, giving $10 \times 9 = 90$ possible communication paths, but we must halve this because each path will have been counted twice, once in each direction.

SAQ 2

Suppose you were able to give an accurate estimate of the work content of a project at the start of the project. Give one reason why the project could take more than this estimate. Give one reason why you might be motivated to propose a lower estimate.

ANSWER.....

A project might take more than the estimate because the requirements might change, creating further work. You might have been tempted to propose a lower estimate in order to secure the business.

SAQ 3

When you organise a project team by subsystem, as in Figure 3, what considerations should you take into account in allocating staff to a subteam? Assume that you are working within a matrix organisation.

ANSWER.....

The primary need is to make sure that each subteam has the skills and knowledge necessary for developing its particular subsystem. The use of a matrix organisation makes this easier.

2.2 Quality management

In *Unit 11* we looked at product quality. Quality was described as being fit for purpose, that is, of sufficiently high quality to meet the requirements and expectations of the customer, and you saw a number of ways of characterising and measuring software quality. In Section 4 of this unit we shall see how to marshal the product quality techniques described in *Unit 11* so that together they assure a quality final product. This is known as *quality assurance* (QA) or *quality management*. In particular, in Section 4 we shall look at quality management systems (QMSs), the ISO 9000 family of quality standards and the *capability maturity model integration* (CMMI). Another QMS that you may come across is *total quality management* which has many good ideas, though as a general approach to quality assurance it has not proved a lasting success. The concept originated in the US, but achieved its global popularity via Japan. It takes a customer view of quality. This has been a view that has been very strong in Japan, where customer satisfaction has been placed higher than immediate profit. In the total quality management TQM movement, a frequent claim is that 'quality is free', yet adding procedures to assure quality seems to add cost. There are two answers to this apparent paradox.

- In the long term what matters is having customers, and it is no good making a large profit from today's customers if they do not come back tomorrow. Extra effort in supplying quality to customers today, with reduced profit margins, is recovered later when the customer returns again and again.
- Making visible a concern for quality also makes visible the processes and procedures associated with assuring the quality, so a new source of expenditure is seen. What is not so visible are the savings that are made as a consequence of this.

Thus the critically important concern of total quality management is customer satisfaction. This leads to the concept of a *quality chain*.

To illustrate this, suppose you buy a product from a shop, but when you get it home you discover that it has been soaked at some time in the past and doesn't work. You return it to the shop and complain angrily to the sales assistant for selling you defective goods. Is it his fault, as he should have noticed the damage before he sold it to you? Is it his manager's fault as he unpacked the goods from the supplier, noticed the damage and was too busy to do anything about it? Is it the supplier's fault, where the flood occurred which damaged the goods? Regardless of initial fault, the result was a

defective package being passed on down the line, with a succession of people prepared to accept and pass on the faults until the customer complained. We see here not only the acceptance of failure, but a sequence of such acceptances. In a *quality chain*, there is a succession of stages all of which are responsible in part for the final quality delivered to the customer.

In TQM, failure is not acceptable at *any* stage of a quality chain. You can think of a quality chain as a sequence of supplier–customer relationships: in the example, there are three such relationships, between the supplier and the store manager, between the store manager and the sales assistant, and between the sales assistant and you. In the TQM approach, the supplier at each stage takes a customer view of quality, in which the provision of defective goods or services to the customer is unacceptable. Thus in the example, if total quality management were in place, together with an appropriate process for detecting defective goods (such as inspection of goods prior to despatch at each stage), the defective software would never have left the supplier, let alone reach the shop's stores or your home.

There are a number of ways to make the TQM approach a success. One that has received wide publicity is *quality circles.* Quality circles are an important, albeit small, aspect of total quality management. A quality circle is a way of involving everybody in the production of quality products. Quality should not be simply the goal of management. A quality circle is a group of four to ten volunteers who meet regularly, perhaps once per week, to identify, analyse and solve their work-related problems. Note that it is the work-related quality problems that are discussed, not the work of individuals or the individuals themselves. Note the critically important difference here between quality circles and the product quality reviews and inspections described in *Unit 11*: reviews and inspections focus on a work attributable to an individual; the focus of quality circles is much broader, and reflects a common social value of never criticising (shaming) an individual.

Quality circles, as with the whole of the TQM approach, take the view that we are all involved with the work of the company, and can all influence the quality of the product. It is not somebody else's problem or fault; we are all responsible. In many instances, this attitude requires a change in company culture, because quality and service must pervade all aspects of work.

SAQ 4

Consider the waterfall model of the software development process as a quality chain. Who are the customers for the design activity and what must be done to gain their satisfaction?

ANSWER.....

There are two sets of customers: the requirements engineers, who will be satisfied if the design implements the requirements determined by them, and the programmers, who require a design that is free of defects and is easy to implement. Both sets of customers have a view of what a defect is, and require the design to be defect free, from their point of view, to be satisfied.

2.3 Configuration management

As a system is developed, the number of items produced, from UML diagrams through to program code, continually increases, eventually numbering many thousands. Items are developed by one person or team, and then used by others in their development

work. Problems are discovered and items are changed, and then changed again, so that in the end each item exists in several versions. Changes to a version of an item can happen in parallel, and need coordination. Versions of items are interdependent, and it is very easy to lose track of which version of which item works with which version of some other item. Even when there are relatively few items chaos can still ensue. Preventing this chaos is a management task, known as *configuration management*, which is discussed in detail in Section 5.

SAQ 5

What basic problem does configuration management aim to solve?

ANSWER.....

Configuration management aims to solve the problem of keeping track of all the successive versions each item goes through, as well as the interdependencies between them, avoiding the possibility that developers will waste effort by working with wrong versions, and ensuring that when the product is released only the correct version of each part is included, so that the product will work properly.

Exercise 1

What are the primary concerns of the manager of a software development project?

Solution.....

Although management can be divided into project management, people management, and quality management, these are each carried out to ensure that the software is delivered:

- on schedule;
- within budget;
- to specification.

These factors are the critical concerns in managing a software development project.

Exercise 2

What skills does a project manager need to possess to carry out the responsibilities set out in our solution to Exercise 1?

Solution.....

Just being a good programmer or analyst is not enough to be a good project manager. The project manager will need to be skilled in the following areas.

- *Effective people management* The project manager must be able to identify the skills and expertise required to complete the project, and then recruit the staff. And, perhaps more importantly, they must be able to motivate and develop these people, to their full potential, to benefit not only the current development project, but also the whole organisation over the longer term.
- *Planning* Any kind of construction project always requires careful planning. The project manager needs to plan each activity of the development so that resources can be made available when required, and not too early or too late. The project plan the manager constructs must also give confidence to the client that the project is being properly managed.

- *Organisation and monitoring* The project manager must organise the human resources available into an effective team or group of teams. Having planned the work, the use of resources with respect to the plan needs to be plotted and any potential problems of slippage or overspend need to be identified early and remedial action taken.
- *Customer relations* In many projects, liaising with the clients and users is a major activity in itself. Clients will need to be reassured that the money they are investing in a new software system will bring the expected return on investment. Users will want to be reassured that the software will support their needs. On many projects, requirements are constantly being changed and new ones introduced, and managing this is part of the project manager's job.
- *Technical leadership* In many smaller projects, the project manager will be the person consulted for advice in devising and selecting solutions to technical problems. The project manager will not be expected to become involved in the details of these solutions, but must be able to spot the winners and losers. Alternatively a different technical leader or 'authority' might be appointed, perhaps called the 'system architect' or similar.

2.4 Summary of section

In this section you have been introduced to process quality and its support through project and people management, quality management and configuration management.

You saw that project management is more concerned with controlling costs and making sure that the software is produced on time, and people management is more concerned with enabling staff to work successfully.

You saw that quality management is a general need, and looked briefly at one broad approach: TQM. Product quality and quality assurance were discussed and the need for configuration management to keep track of the range of versions and deliverables was noted.

3 Project management

In Section 2 you saw how process quality is determined by a number of management activities. In this section we shall look in more detail at aspects of one of these management activities: project management, which is the process of planning a project, estimating the work content, assigning that work to people and scheduling when it will happen; then monitoring the progress of that work and taking corrective action if something does not go according to plan.

3.1 Overview of project planning and control

A project manager has many responsibilities. Most important among these are the ones relating closely to maintaining the agreement of the customer to the development being done on their behalf, namely:

- liaising with the customer (who may be internal or external to the organisation);
- ensuring that any contractual obligations are fulfilled.

Others, associated with quality management, are taken up in Section 4:

- producing a quality plan;
- making sure appropriate tools are used;
- making sure that the lessons learned on other projects in the organisation feed into this project and that this project's lessons are passed on to others.

The remaining responsibilities are all related to laying out the work to be done, and then ensuring that the work gets done. These each translate into one or more tasks that the project manager must perform, summarised as follows:

- analyse and manage risk;
- select people for the project;
- organise project teams;
- estimate work content and cost;
- schedule work;
- assign tasks to teams and team members;
- monitor the progress of the project;
- keep the project on track.

Selecting the right people and organising them into teams is normally done at the start, though these tasks may need revisiting later in the project. These two tasks were covered in Section 2. The rest of the tasks are covered in the subsections below.

3.2 Risk analysis and risk management

No software development project is free from risk, and one of the key activities for a project manager is identifying and managing risks. The project manager needs to consider the probability of an unwanted event occurring, and its associated cost. It is important for the project manager to understand both the probability and cost so that most effort goes into monitoring the important risks: those with high probability and/or high associated cost.

We can categorise risks into *project*, *technical* and *business* risks.

Adapted from Pressman (2000).

- *Project risks* are those risks directly associated with the management of the project, for example budgetary, scheduling, personnel (staffing and organisation), resources, customer and requirements risks.
- *Technical risks* are those risks concerned with the development and technical aspects of the project, for example design, implementation, interfacing, maintenance and operating platform risks.
- *Business risks* are those risks that can impinge on the project but which derive from the client and user environments. They are often far harder to identify as they may come from areas outside the project manager's control. They include, for example, risks associated with changes in policy in the client's department, changes in the client organisation, changing client priorities, and changes in legislation. Another common and important business risk is the risk of the system working successfully at a local level but causing problems at a wider organisational level.

This is just one way that risks can be categorised. We use categorisation to make the process of risk identification, monitoring and management more tractable. One categorisation scheme may be as good as any other — the acid test is whether it helps in the process of risk management.

The project manager needs to identify as many of the potential risks to a project as possible, although it is unlikely that all risks will be identified (another risk!). If the project manager is experienced, many risks will be known at the outset. One technique to identify others is to hold a 'brainstorming' session with the client and the development staff. It is not possible to present a generic list of project risks — each project's risks will be different and will depend on the nature of the project, the relationship with the client, the expertise of the development staff and the client staff, and the maturity of the operational environment. Risk identification is not a trivial task and a categorisation such as that described above may assist in the process.

Having identified the potential risks, the next thing to do is assess their seriousness. We need to estimate the likelihood (probability) of the risk event occurring, and then we need to estimate its impact (the consequent cost should the risk materialise). The method used will depend on the nature of the project. For some projects, a fairly informal approach will be sufficient. For example, we could simply score the likelihood of each risk as 'high' or 'low' and its impact as 'high' or 'low', and only worry about the high-risk, high-cost events. Alternatively, a more formal approach is to estimate a numerical value for each likelihood and impact. Then, if we multiply the estimated likelihood of the risk by the estimated impact if the risk happens, we get an *expected cost* — if this is high, then we must do something to try to prevent the corresponding risk event occurring.

It is very unlikely that all the risks to a project will be identified at the outset; some will not be identified until the development is under way. Some may not even be identified unless the risk event actually occurs. The point is that unless a conscious effort is made to identify risks early on, the project manager is likely to end up reacting to problems rather than anticipating them.

Having identified the important risks, what do you do? Four strategies are possible.

Risk avoidance

This strategy presupposes that the risk is entirely within the management bounds of the project manager and that the project manager has the resources and authority to deal with it. It is often the best strategy — prevent the risk event from happening in the first place. An example might be: 'My staff are not fully up to speed with that new visual

programming environment for Java, so we'll use C++ instead and traditional tools, with which they are familiar.' Of course, this strategy might well introduce a new risk, since avoiding the original risk will result in the loss of any benefits that may have accrued from taking that risk — in this example, using Java. Risk management strategies are invariably a balancing act, requiring that project managers exercise their experience and judgement.

Risk retention

If the risk is seen as low probability and low cost, in other words the risk is unlikely to occur and if it did the effects would be minimal, the project manager might recommend that the risk be accepted. This would be the strategy to adopt if the costs of avoiding the risk were significantly higher than the costs of the risk event occurring. Under some circumstances you may find that you have to retain a risk because there is absolutely nothing you can do about it — none of the other risk strategies can be applied. An example here would be the development of a leading-edge product such as a computer game, where the only way to obtain the visual effects required means using prototype software that may fail during use and lead to the failure of the game product.

Risk reduction

This is probably the commonest strategy. It is unlikely that the project manager will be able to eliminate risk entirely, but controls and countermeasures can be put in place to reduce the likelihood of a risk and to reduce its impact should a risk event occur. An example of risk that can be reduced is the risk of a bespoke system not being accepted by its intended users and becoming 'shelfware'. This risk can be reduced by involving users in the development process, in what is known as 'participative development', as members of review panels, or even as software designers. Of course there is still residual risk of rejection, particularly if the participating users do not properly represent the whole community of users. In addition, this strategy might introduce another risk: that the process of developing the system is slowed down by having to train the users in software development practices.

Risk transfer

This does not mean that the project manager abdicates responsibility for the risk, but that the costs resulting from an occurrence of the risk event are passed on to a third party. An example is outsourcing parts of the product development to a professional software house on a fixed-price basis.

SAQ 6

Suggest a way risks to a software development project might be identified and categorised.

ANSWER.....

They could be identified by holding a brainstorming session and using some simple categorisation of risk to help focus the search. A suitable categorisation would be: project risks, technical risks and business risks.

SAQ 7

A project has identified a number of risks, and is considering the following strategies to contain the risks:

(a) to continue developing in Java, despite learning the next release of the Java development environment is quite different from the current version;

(b) to put a penalty clause in a software procurement contract so that a failure by the software supplier to deliver acceptable software leads to complete recovery of all payments made, plus an additional damages payment to cover consequential loss;

(c) to send all software engineers on a UML training course;

(d) to not use Java, because the software engineers have no experience of Java.

Say what kind of risk containment strategy is being used (from avoidance, retention, reduction and transfer) in each case, and discuss the appropriateness of the strategy.

ANSWER.....

(a) Risk retention: either the current development environment can be retained until the project is complete, or if not, the retraining time can be tolerated.

(b) Risk transfer: the supplier now carries the risk.

(c) Risk reduction: using UML will be less risky if everybody has a common understanding of current best practice in using UML.

(d) Risk avoidance. Note that one could have used risk reduction by training the software engineers on Java. In addition, not using Java could bring its own risks, such as using outmoded technology.

SAQ 8

A project manager will want to monitor those risks that are of significant concern. Which kinds of risks do you think should be most closely monitored?

ANSWER.....

At the outset of the project the project manager will be most concerned about those risks identified as high probability and high cost. However, the project manager will also need to monitor not only these significant risks, but also those risks which at the outset seem insignificant but whose probability and costs may change as the project develops, in case these risks become significant. Software development is an extremely dynamic process and situations, especially with regard to risks, can change suddenly.

3.3 Estimation

Before planning a project, one must have some estimate of the resources required and problems to overcome during the life of the project. This subsection presents some estimation techniques. Estimation needs to be carried out regardless of the choice of software development process. There are several well established methods for estimating that are applicable to the development of object-oriented systems.

The experience of the software industry is that to develop software requires, on average, roughly one person-hour of effort to deliver two or three lines of finished program code. This includes all the effort from requirements analysis through coding to testing and delivery, and does not depend on the programming language used or whether it is object-oriented or not. The variability in the amount of effort required

comes from the type of product being developed, its size and complexity, and in some measure from the varying ability of individual software engineers. For very large and complex systems, productivity could be as low as 4 lines of code per person-day of effort; for really small and simple systems, it might exceed 10 lines of code per person-hour of effort.

How has the software industry learned this? By recording the total effort used during a project, and after the event finding out the size of software produced and doing some simple arithmetic. But what we want is to be able to predict the effort required before even one line of code has been produced. We want to quantify something about the project and the system to be built at the start, or as early into the project as possible. Based on these estimates, we want to predict what resources and time will be required, and then update these predictions as we learn more about the project.

There are two interrelated project or system factors that need to be considered: *system size* and *system complexity*. **System size** can be measured initially by the number of functions, the amount of data and the number of users. At completion of the project, it can be measured by the number of lines of code (LOC; or KLOC for thousands of lines of code) and the volume of data. As size grows, there is a danger that, in order to grasp the nature of the problem through abstraction, the level of abstraction may increase, with the consequence that detailed understanding decreases and assumptions start to be made about the system. This can cause initial estimates of the size of large systems to be inaccurate.

System complexity is often a subjective measure and relates to the interdependencies between elements of the system. For a given size of system, complexity will vary depending upon the type of system. Real-time embedded systems are generally thought to be more complex than data-processing systems.

A typical estimation method consists of the following steps.

1 Estimate the system size, using what you know about the system at the point you are making the estimate.

2 Adjust the size to take account of the system complexity and various other 'cost drivers' such as the experience of the software engineers.

3 Convert the estimated LOC or other size measure into effort required.

4 Estimate the optimal time to complete the development.

5 Estimate other properties of the software, such as expected defect rate and quality.

Note that the relationships between effort and size and between effort and optimal time are not linear, so that, for example, doubling the size means that you must more than double the effort. The main reason for this is that having more people involved increases the amount of communication required.

Estimates are just that — estimates, which can be wrong. Experienced project managers know and accept that their initial estimates will often be way off. The point is that the estimation process is a continuing activity, with the project manager revising the estimates on a regular basis. One way of doing this is to monitor the LOC produced at various stages and to use these as inputs to a new round of estimation. Also, as the project progresses, so the clients, users and developers improve their understanding of the requirements and the system being built, and with this greater understanding comes greater precision in estimating.

All estimation is based in one way or another upon experience from previous software development. The estimation method must be 'calibrated' against this experience. Sometimes the calibration is against the industry at large, and sometimes against experience within the organisation concerned. In practice, both are needed: the

industry experience to get a large range of projects and enough data to make the calibration statistically meaningful, and local experience to make the calibration true for local conditions. As we look at the four different estimation methods below, we shall see how this prior experience is brought to bear.

Estimation by analogy

If the software we wish to build is similar to software we have built before, then we can directly use the experience from that previous occasion or occasions. It must be assumed that we have access to records of how much that earlier development cost, what effort was used, how large the software was, as well as a comprehensive description of the software so that we can assess how well our new project matches the previous one. If the new development is different in small ways, perhaps with a few different functions or a different target platform, we might be able to extrapolate from that earlier experience. Or it could even be that, although functions are different, the overall form and nature of the software is analogous, and we can still carry forward the previous experience. For example, customer service systems for online travel agents and mobile telephone companies will have similar structures, even though the products being supported, the interfaces to external systems, and the types of questions being asked will be very different.

All of these approaches are generally referred to as *estimation by analogy*. Informally this can be carried out by asking several experts to draw upon their experience and give estimates, and then to seek convergence of the individual estimates. One such method is the Delphi technique, which uses a facilitator to collect estimates together with the reasons, which are then distributed anonymously for another round of estimation.

Estimation by analogy can be made more formal by collecting descriptions of systems and their associated costs and other parameters to form a database. If possible a range of estimates is given for each system, with (at least) a best-case and worst-case pair, and perhaps a most likely value as well. A new estimation problem is then matched against this database. There are lots of problems with this formalisation, since any written description is necessarily only partial and may miss out key features. It can work, though perhaps only for relatively routine systems that are produced repetitively.

However, if a system is similar to some previous system, we have other ways of handling these new systems, through reuse. We saw this in *Unit 10*, and it could be that exploration of analogies leads to product lines and components, and the accompanying reductions in costs.

Estimation by work breakdown

An alternative approach is *estimation by work breakdown*, where the work to be undertaken is broken down into smaller chunks that can then be estimated by analogy. This is usually done by decomposing the system into subsystems and components that seem similar to previous systems, with consequential opportunities for reuse or at least estimation by analogy. We might continue the decomposition (really as part of the design stage) until very small units of software are identified that can be more accurately estimated by an experienced software engineer. Other aspects of the work, such as testing and management, can then be added in as some percentage of the other estimates. The separately estimated parts are then added to give an estimate for the whole. Estimation by work breakdown is particularly useful for novel systems where little or no prior experience is available.

The advantage of estimation by work breakdown is that work breakdown is going to be undertaken anyway, in order to assign the work to teams and individuals and thus be able to schedule the work and produce a plan. However, the actual breakdown

required may only be available well into the development process, so that estimation by work breakdown can not be used for early estimates.

3.4 Function point analysis

FPA has become so well established that an International Function Points User Group (IFPUG) was established in 1986; you can see its website by clicking on the IFPUG hyperlink on the web page for this unit.

Function point analysis (FPA) is a well established technique for estimating the size and complexity of a software project, and hence for providing a measure of how much effort will be required to develop the software.

The idea of *function points* goes back to the mid-1970s, when it was realised that you can assess the size of systems in terms of the functions they perform. First count the number of inputs to a system, the number of outputs from the system, and the number of data files or elements stored in the system, weight each of these by an assessment of its complexity, and then add them all up to arrive at a size measure for the software in terms of function points. You then translate from function points into effort or lines of code, based upon experience of previous projects. The assumption is that each function point corresponds to the same amount of effort.

This idea transfers well to object-oriented approaches. One gains an understanding of the software functionality by producing a use case diagram and a class diagram during the requirements analysis stage of the development process, and these diagrams provide what you need to obtain a function point estimate for the project. The following is a highly simplified account of how to proceed. We are concerned with the principles, and not providing details of a practical method.

Use case models and class models were discussed in *Units 3* and *5*.

This account is based on a paper by Fetcke, Abran and Nguyen (1998). The authors refer to the IFPUG method for non-object-oriented approaches, and map object-oriented approaches to that, distinguishing different kinds of object.

First, working from the use cases that model the requirements, every use case that is connected to an actor (or maybe several actors) outside the system boundary counts as an element of functionality. This is true regardless of whether the actor is a human or another system. Use cases that «`extend`» use cases in this first set (use cases connected to an actor) are also counted. Note that these rules mean that some use cases that are connected only internally (not directly connected to an actor) are not counted; you saw examples of this in *Unit 3* where one internal use case has an «`include`» dependency, and another use case has an «`extend`» dependency. Call the count of uses cases found using these rules F_U.

Secondly, we count the classes in the requirements class model. Every class contributes to the total, with no exception made if, for example, a class is abstract or if it defines a type of object that only ever occurs as part of something else. Call this count F_C.

Note that the weights W_u and W_c are average weights, taking into consideration the complexity of the collection of use cases and classes.

Function points are then estimated from the counts by weighting them with adjustment factors — call these W_U and W_C respectively — depending on one's judgement of the complexity of the use cases and classes. The actual weights are usually somewhere in the range 4 to 7 for use cases, and 7 to 15 for classes. We end up calculating the function points (FP) as:

$$FP = F_U \times W_U + F_C \times W_C$$

It would then be common practice to adjust the FP value for the complexity of the actual system to be built. The FP value is adjusted as follows:

ΣF_j means the sum of the F_j values.

$$FP_{adjusted} = FP \times (0.65 + 0.01 \times \Sigma F_j),$$

where the F_i are fourteen factors, each with an (integer) value in the range 0 to 5 obtained in answer to each of the following questions.

1 Does the system require reliable backup and recovery?

2 Are data communications required?

3 Are there distributed processing functions?

4 Is performance critical?
5 Will the system run in an existing, heavily utilised operational environment?
6 Does the system require online data entry?
7 Does the online data entry require the input transaction to be built over multiple screens or transactions?
8 Are the master files updated online?
9 Are the inputs, outputs or enquiries complex?
10 Is the internal processing complex?
11 Is the code designed to be reusable?
12 Are conversion and installation included in the design?
13 Is the system designed for multiple installations in different organisations?
14 Is the application designed to facilitate change and ease of use by the user?

The answer to a question determines the value to be given to the corresponding factor according to the following rating system: 0 = no influence, 1 = incidental, 2 = moderate, 3 = average, 4 = significant, 5 = essential. Of course, whether a particular factor is regarded as being of no influence or essential is highly subjective, and organisations need to develop criteria for deciding how to rate the factors. It is important that the method of producing the ratings is well understood, and that the underlying assumptions are recorded.

Note that if every factor were able to be set at 2.5, the adjustment would have no effect.

Where do all these weights and factors come from? Originally they were derived from the analysis of a large number of projects, so they are experience-based. As each organisation is different because of local development practices and experience of particular system types, many organisations tune these weights and factors to suit their particular circumstances.

Converting function points into effort is typically done by first converting them to lines of code for the particular language being used. Again this is based on experience, and the belief that a function point produces the same volume of code on average, regardless of where the function point has arisen. Table 1 shows one such mapping, based on the industry experience.

Table 1 Mapping from function points to lines of code

Programming language	LOC per FP (average)
Assembly	172
Ada	154
C	148
C++	60
COBOL	73
Java	60
Smalltalk	35
SQL	39
Visual Basic	50

There are many different sources for mapping lines of code to function points. The numbers in this table come from QSM Inc. See http://www.qsm.com/FPGearing.html (Accessed 12 March 2008).

Converting the resulting KLOC (1000 lines of code) into effort and time could be done by a number of methods; the one we give here is COCOMO, which is explained below.

COCOMO

See Boehm (1981).

Given an estimate in KLOCs for the size and complexity of a software project, we still need to find the effort involved in and the expected duration of the project. One way of doing this is to use COCOMO, the COnstructive COst MOdel, developed by Barry Boehm (pronounced 'Beem') over many years based on his experience with large defence contracts. In the early 1990s it was realised that Boehm's original COCOMO assumptions no longer applied to systems that were being developed then, and a project was started to update the method. This new method has come to be known as COCOMO II, with the original method now being known as COCOMO 81.

For COCOMO II developments, click on the COCOMO hyperlink on the web page for this unit.

What COCOMO gives us is a simple means of converting from code size (in KLOCs) to effort in person-months and to optimal project duration in months. Optimal project duration is a prediction of the time the project will take in months, assuming we have E person-months of effort available. This concept is further discussed below.

As was mentioned earlier, effort is not proportional to code size (that is, their relationship is not linear), and some measure of diseconomy of scale comes in. COCOMO's formula for effort E (measured in person-months) in terms of code size is:

$$E = a(\text{KLOC})^b,$$

The notation $a(\text{KLOC})^b$ means a times the KLOC value raised to the power of b. The built-in Windows calculator will do this calculation for you. Open the calculator found in the `Start->Programs->Accessories` menu, Choose `Calculator`, change the view to `Scientific`, type in the KLOC value, press the `x^y` key, type in the b value, and then press =. Press the × key, type the a value, then press = for the answer.

where a is the nominal productivity, in person-months, to produce 1000 lines of code (1 KLOC) and b determines the degree of diseconomy of scale (if it is greater than 1) or economy of scale (if it is less than 1). So, for example, a million lines of code (1000 KLOC) is estimated as requiring proportionately twice as much effort if $b = 1.1$ and almost four times as much if $b = 1.2$.

It was also mentioned earlier that project duration is not proportional to effort. The COCOMO formula for optimal project duration D (in months) in terms of effort is:

$$D = cE^d,$$

where c is the basic duration, in months, per person-month and d gives a measure of the non-linearity. Larger projects should take a proportionately shorter time (for example by involving more people working simultaneously), so typically d is less than 1.

(Note that COCOMO gives the effort in person-months rather than person-hours. For these estimates of effort and duration to make sense, the project manager needs to be clear what is meant by a person-month on their project and to make any necessary adjustments to the model so that it gives appropriate estimates.)

The simplest form of COCOMO 81, known as Basic COCOMO, recognises three classes of software project:

The names *organic, semi-detached* and *embedded* are Boehm's own terms; don't worry about their origin, just remember that we have three classes of software ranging from the relatively simple to the complex.

- *organic:* relatively small, simple projects;
- *semi-detached:* intermediate projects in terms of size and complexity;
- *embedded:* complex software projects, such as a flight-control system for an aircraft.

Table 2 gives the parameter values for these three classes used by Basic COCOMO.

Table 2 The parameter values for Basic COCOMO

Software type	Parameters			
	a	*b*	*c*	*d*
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

These parameter values were based on the analysis of many software projects with which Boehm had been involved. As with function point weights and factors, many organisations tune these parameter values on the basis of their own software development practices, based upon actual project costs accumulated over many years.

COCOMO II uses the same method as Basic COCOMO to calculate effort, using the formula:

$$E = a(\text{KLOC})^b,$$

but now the exponent b is subject to adjustment by five scale factors W_i using the formula:

$$b = 0.91 + 0.01 \times \Sigma W_i$$

These scale factors capture the Basic COCOMO ideas of organic, semi-detached and embedded projects, as well as other influences. Then the effort is adjusted using effort multipliers, as for Basic COCOMO, as:

$$E_{\text{adjusted}} = E \times \Pi EM_i$$

To compute ΠEM_i, which stands for *product of* EM_i, you multiply together all of the effort multipliers EM_i.

At different stages of the project, a different set of effort multipliers is used: seven at 'early design' and 17 at 'post-architecture', later in the development process, when more is known about the system development.

Example of estimation

Consider the problem of developing a library system discussed in *Unit 3* and *Unit 5*. Analysis of the problem produced a use case diagram (shown here as Figure 4), to which we have added a simple class diagram — see Figure 5. We shall use these diagrams, together with function point analysis and COCOMO, to estimate the effort and time required to develop a software system for the library.

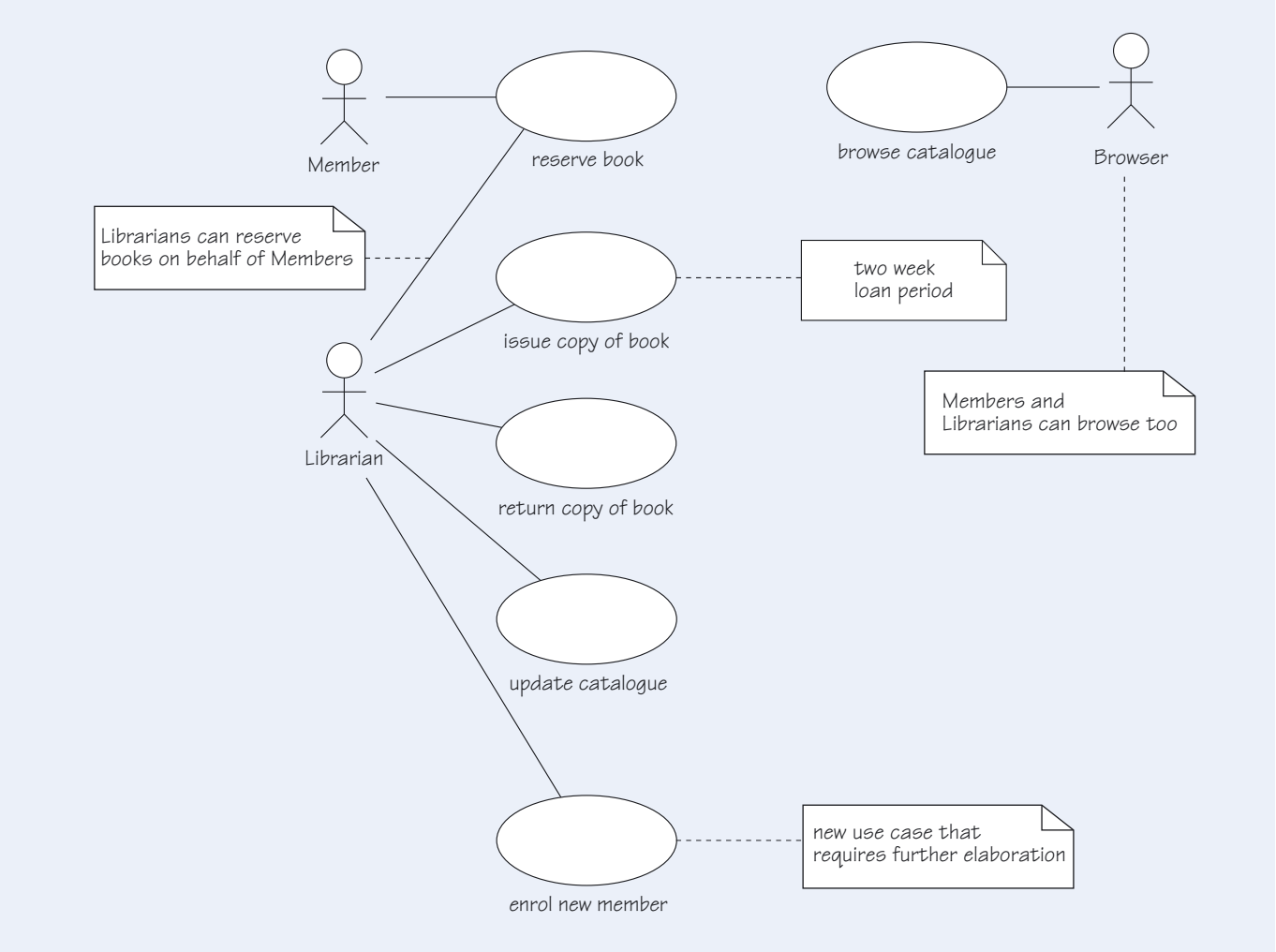

Figure 4 A use case diagram for a lending library

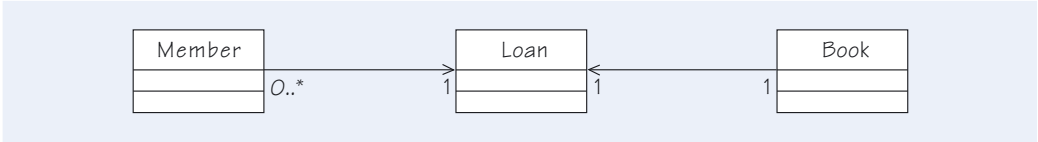

Figure 5 The classes for a lending library

Step 1: apply object-oriented function points

We can take all the actors to be human users external to the system, so the system boundary lies between the actors and the use cases. There are six use cases. The *Look Through Catalogue* use case is simple, in that it requires only read access to the catalogue to complete. However, it is not that simple since it will have embedded exception-handling for cases of wrong data and access of the catalogue while it is being updated. The other five use cases could be really quite complex, involving searches of and modifications to the library catalogue, incorrect data and missing books, and so on. So we shall weight all the use cases high, with a weight of 7, to give an FP contribution of 42 so far.

The class model has only three classes, and they are relatively simple as far as we know. So we shall weight them all at 9, to give an FP contribution of 27.

This then gives an FP value of 42 + 27 = 69.

We shall now adjust the FP value for complexity of the operational system using values for the fourteen factors as shown in Table 3.

Table 3 Values for the complexity factors

F_i	F_i value	Reason
1	4	Backup and recovery is important, and could be daily.
2	4	Data communications are required, but only over a LAN.
3	0	There are no distributed processing functions.
4	1	Performance is not critical, other than for human–computer interaction (HCI) reasons.
5	0	A dedicated system will be used.
6	5	All data entry is online.
7	1	Actual data input will be from a single screen.
8	1	The records of books in the library could be updated overnight.
9	5	The inputs, outputs and enquiries are complex.
10	0	The internal processing is relatively simple (see the interaction diagrams in *Unit 6*).
11	0	Code will not be designed to be reusable.
12	0	A manual system is currently in use, so no conversions are necessary.
13	0	There will be a single installation.
14	4	Usability is important.
TOTAL	**25**	

The factors in Table 3 in the table are not 'universal truths', but instead are informed estimates. You might ask 'how do we obtain these factors?' We estimate them as best we can, taking into account any information or insights that we possess, and perhaps erring on the side of caution by taking a hgher rather than a lower value for any factors we feel very unsure about.

Inserting the total, 25, into the equation for $FP_{adjusted}$ gives:

$$FP_{adjusted} = 69 \times (0.65 + 0.01 \times 25) = 69 \times 0.9 = 62.1$$

If we now turn to Table 1, relating function points to lines of code, this then suggests 60 lines of Java code per function point, so that we would need to develop approximately 3750 lines of Java code to implement the library system.

Step 2: apply COCOMO to obtain effort and duration

We shall apply Basic COCOMO, viewing the library system as an exceptionally simple organic system. We thus take figures from Table 2, to obtain:

$$E = 2.4 \times 3.75^{1.05} \approx 9.6 \text{ person-months.}$$

We do not have enough information to be able to adjust this value by use of effort multipliers, so we will use this value for the effort.

The optimal duration is then:

$$D = 2.5 \times 9.6^{0.38} \approx 5.9 \text{ months.}$$

Note the earlier comment about being clear what is meant by a person-month.

D predicts the time the project will take in months, assuming we have *E* person-months of effort available. If we add resources, the duration will increase because of communications overhead. If we subtract resources, the duration will increase because of work load.

SAQ 9

What are the basic ideas underlying function point estimates, and how are these applied to estimation of object-oriented software development?

ANSWER.....

Function points are units of functionality of a system such that each function point requires the same quantity of code to be produced. Functionality is known early in development, during requirements analysis. In object-oriented development, functionality is determined by the use cases and classes, so these are used for estimation purposes.

SAQ 10

If you have an estimate of the size, in KLOC, of a software system that you intend to develop, why can't you then simply divide the size in KLOC by a single productivity figure of KLOC written per person-month to obtain the effort?

ANSWER.....

In developing large software systems, two further factors are involved.

- Complexity: productivity is higher for simple systems such as database systems, and lower for complex systems such as embedded real-time systems.
- Diseconomy of scale: usually larger systems use proportionately more effort due to the need for internal communication between members of the project team.

Both of these mean that simply dividing by a single productivity figure will not suffice.

SAQ 11

How do we incorporate our judgement of the potential complexity of a system into the process of estimation using Basic COCOMO?

ANSWER.....

Complexity is first of all used to select the actual parameter set to be used, depending upon type of system — organic, semi-detached or embedded. After this, complexity may also be used to determine a number of effort multipliers, which are used to scale the initial estimate of the effort.

Work breakdown and scheduling

Once we have estimates for the project effort and duration we can begin to produce a detailed project plan. We need to address two questions.

- What proportion of the overall effort should be devoted to each development activity, such as analysis, design, coding and so on?
- How will different activities be scheduled across the lifetime of the project?

Pressman (2000) gives the following guidance figures for the percentage of the total effort required for various activities:

- project planning and management, 2 to 3 per cent;
- requirements analysis, 10 to 25 per cent;
- design, 20 to 25 per cent;
- coding, 15 to 20 per cent;
- testing, integration and debugging, 30 to 40 per cent.

These draw upon general experience of the industry, and can be useful in determining a resourcing profile for the project. Generally speaking, a project (or an iteration of a project when using agile or spiral methods) starts with a relatively small number of people, then builds up to a peak number around coding and testing, tapering off after that through testing and into support for the software following delivery; the shape of the staffing curve depends on the process model in use. The skills required also change: it starts with analysts and designers, then moves on to a preponderance of coders and testers while retaining core analysts and designers. We can view this as general guidance for the whole project, or as guidance for individual subsystems.

What we really need is a breakdown of the work required. We need to decompose the overall task of developing the software into many much smaller *activities*, which are well defined, can be assigned to individual teams or persons and can be individually scheduled (allocated dates when the work will be done). The difficulty is that any such decomposition will be notional, since the design upon which it must rest will not yet have been done. We aim to end up with a number of activities for which we have work-content estimates. Ideally these activities should be quite small, of the order of 5 to 20 person-days of effort.

We also need to have some idea of the order in which the activities should be done, and of their interdependencies. These interdependencies are important since some activities cannot start until some preceding activity or activities have been completed, or at least have progressed far enough along the road so that the necessary information can be passed on.

Identifying the activities, their work content and their interdependencies is only the start. It then remains to juggle things until the activities can be scheduled in a sequence that honours their interdependencies and keeps the people on the project busy without overworking them. This is known as **resource balancing** or **resource levelling**. We end up with a **project plan**, defining what work will be done, when and by whom. Included in this plan will be a number of project **milestones** — the points where key deliverables are completed, reviewed and then 'frozen'.

We can represent a project plan in many ways. Two commonly used ways are the following:

- **PERT charts**, which show the activities as boxes with lines joining them to indicate the interdependencies and flow of critical information;
- **Gantt charts**, in which the horizontal direction represents time and the vertical direction represents activities, and which can be set out as tables, the rows of which show when the work takes place, or as bar charts, the horizontal bars of which show when the work takes place.

PERT stands for 'project evaluation and review technique'.

Gantt charts are named after their originator, the American management consultant Henry Laurence Gantt.

Thus PERT charts emphasise interdependencies whilst Gantt charts emphasise the times at which things happen. For software development, the interdependencies are often weak, so Gantt charts are preferred and are most commonly used. However, PERT charts do give us the opportunity to undertake critical-path analysis, as is explained below.

Example of project scheduling

Let us now schedule the development of the library system that we estimated earlier as requiring 9.6 person-months of effort to design, implement, and test the system, over a duration of 5.9 months.

We assume that most of the project planning and requirements analysis has been done, though there will still need to be time allowed for project management and some residual requirements analysis. The effort expended on project planning and

requirements was 2.3 person-months, so the entire project should take 11.9 person months: the 2.3 person-months already expended plus the 9.6 person-months estimated to remain. Using Pressman's guidance figures above as a guide, we might (somewhat arbitrarily) divide the effort up as follows.

- Requirements (already done): As we already know the effort was 2.3 person-months, we can compute the percentage: 2.3 person-months equals 19% of 11.9 person-months. However, no person-months are included in the schedule because the work has already been done.
- Design (including 5% for residual requirements analysis): 27% of 11.9 person-months, or 3.2 person-months.
- Coding (including some unit testing and debugging): 18% of 11.9 person-months, or 2.1 person-months.
- Testing, integration and debugging: 34% of 11.9 person-months, or 4.1 person-months.
- Project management: 2% of 11.9 person-months, or 0.2 person-months.

Doing the sums, we see that the remaining work correctly totals to 81% (27% + 18% + 34% + 2%), and the effort correctly totals to 9.6 (3.2 + 2.1 + 4.1 + 0.2) person-months.

To avoid working with small fractions of person-months, we shall convert these figures to person-days. Making an allowance for weekends, leave, illness, training and other activities, we shall assume that a person-month consists of 18 person-days. Rounding to integer numbers, this gives us 58 person-days for design; 38 for coding; 74 for testing, integration, and debugging; and 4 for project management; totalling to 174 person-days. Using our experience with similar projects, we now make a first attempt at scheduling these person-days of effort over the six-month duration of the project (rounded up from the figure of 5.9 months obtained earlier, and rounding management time to a minimum of one half a day per month). This process leads to Table 4.

Table 4 First attempt at a project schedule

Activity	Budget (person-days)	Person-days per month					
		1	2	3	4	5	6
Design (and residual analysis)	58	18.0	18.0	13.0	9.0		
Coding (and unit testing/debugging)	38		12.0	10.0	8.0	4.0	4.0
Testing, integration and debugging	74		2.0	8.0	24.0	24.0	16.0
Project management	4	0.5	0.5	0.5	1.0	1.0	0.5
TOTAL	**174**	**18.5**	**32.5**	**31.5**	**42.0**	**29.0**	**20.5**

These activities are still too large to enable us to produce a detailed schedule, but before decomposing them further let's think about people. In terms of the work to be done and the sorts of skills people are likely to possess, what we need are:

- a part-time project manager;
- an analyst/designer/tester who 'owns' the overall system, designs it and then establishes an integration strategy and test plan to ensure that the code produced does what was asked for;
- a programmer/debugger who codes and later debugs the code.

If we assume that the analyst/designer/tester spends 24 days testing, this means that this person is budgeted as requiring 58 + 24 = 82 days of effort. It also means that the programmer/debugger is budgeted as requiring 38 + 74 – 24 = 88 days of effort. As one person-month equals 18 person-days, this means that the programmer/debugger will need to be on the project for almost 5 months.

When we look at the totals, we find that we have a 'bulge' in the fourth month where we need more people than we have. We need 42 person-days and we have 36 (2 people times 18 days) available. We need to smooth the effort out, which we do by pushing effort into the fifth and sixth months and carrying a small amount back to the second month. We now have enough information to revise Table 4 to produce Table 5, assigning effort to people.

Table 5 Revised project schedule

Software engineer	Budget (person-days)	Person-days per month					
		1	2	3	4	5	6
Analyst/designer/tester (D)	82	18.0	18.0	13.0	16.0	8.0	9.0
Programmer/debugger (P)	88		16.0	18.0	18.0	18.0	18.0
Project manager (M)	4	0.5	0.5	0.5	1.0	1.0	0.5
TOTAL	**174**	**18.5**	**34.5**	**31.5**	**35.0**	**27.0**	**27.5**

We can now divide the work to be done into smaller chunks.

The design activities are:

A1 design the classes and their interaction, by developing the interaction diagrams and the full specification of the classes, including the design of the internals of the classes with state diagrams if required;

A2 design the human–computer interaction (HCI) systems from the use cases.

The small residual analysis activity is:

A3 update use cases and class diagrams to maintain consistency with the design.

The test activities are:

A4 specify system tests from use cases;

A5 integrate the system and run the system tests;

A6 specify and conduct the client acceptance tests.

The programming and debugging activities are:

A7 program the classes;

A8 program the interaction systems;

A9 debug the classes;

A10 debug the interaction systems.

This then leads to a possible Gantt chart for the project, a view of which is shown in Table 6.

Note that the process of moving from the level of information in Table 4 to what we find in Table 6 is iterative. When planning tasks, we may find that we must move or reallocate effort to ensure that we have the right people at the right time, which feeds

back into Table 5 and possibly Table 4. What we show here is the result of these iterations.

Table 6 Gantt chart for the project

Activity	Budget (person-days)	Person	Description of activity	Person-days per month					
				1	2	3	4	5	6
	4	M	Project management	0.5	0.5	0.5	1.0	1.0	0.5
A1	30	D	Design classes and their interaction	13.0	11.0	4.0	2.0		
A2	20	D	Design HCI systems	5.0	7.0	5.0	3.0		
A3	8	D	Update analysis diagrams			4.0	4.0		
A4	8	D	Specify system tests from use cases				4.0	4.0	
A5	10	D	Integrate system and run system tests				3.0	4.0	3.0
	50	P				8.0	12.0	14.0	16.0
A6	6	D	Client acceptance tests						6.0
A7	17	P	Program classes		12.0	4.0	1.0		
A8	7	P	Program interaction systems		4.0	2.0	1.0		
A9	7	P	Debug classes			2.0	2.0	2.0	1.0
A10	7	P	Debug interaction systems			2.0	2.0	2.0	1.0
TOTAL	**174**			**18.5**	**34.5**	**31.5**	**35.0**	**27.0**	**27.5**

This Gantt chart view is still not adequate. We should really schedule the work weekly, identify the deliverable(s) for each activity, and determine the milestone dates for those deliverables. We may also want to decompose the programming activities and maybe also some of the design activities. But we do not have space to do that here.

Let us now look at the alternative scheduling method, involving the use of PERT charts, which is based on showing the interdependencies of activities. Many activities are dependent on preceding activities and cannot start until these are complete. Let us look at our project, using simplified interdependencies.

We must decide what activities depend upon other activities. For this project, we assume that A1 and A2 can proceed in parallel, and that A3 and A7 can proceed in parallel, but cannot start until A1 completes. Test specification activity A4 cannot start until A1 and A2 are complete. Once A4 is complete, system test activity A5 can start — but programming activities A7, A8 and A9 must also be complete before A5 can start. A7 can only start after A1 is complete, and A8 can only start after A2 is complete. The debugging activities A9 and A10 proceed in parallel after the system test specification activity A4 is complete. System test activity A5 can only start once A3, A4, A9, and A10 are complete. A6 starts after A5. This is all shown in the PERT chart in Figure 6.

In the PERT chart in Figure 6, a box indicates an activity. The number on the left above a box indicates the earliest working day the activity can start. The number in the middle is the duration of the activity in working days, which is usually the number of person-days of effort from the Gantt chart. However, for task A5 we assume that the designer and the programmer are working in parallel, so the duration is 50 working days instead of the 60 person-days of effort. The number on the right is the start plus the duration, and indicates the earliest working day that an activity can finish.

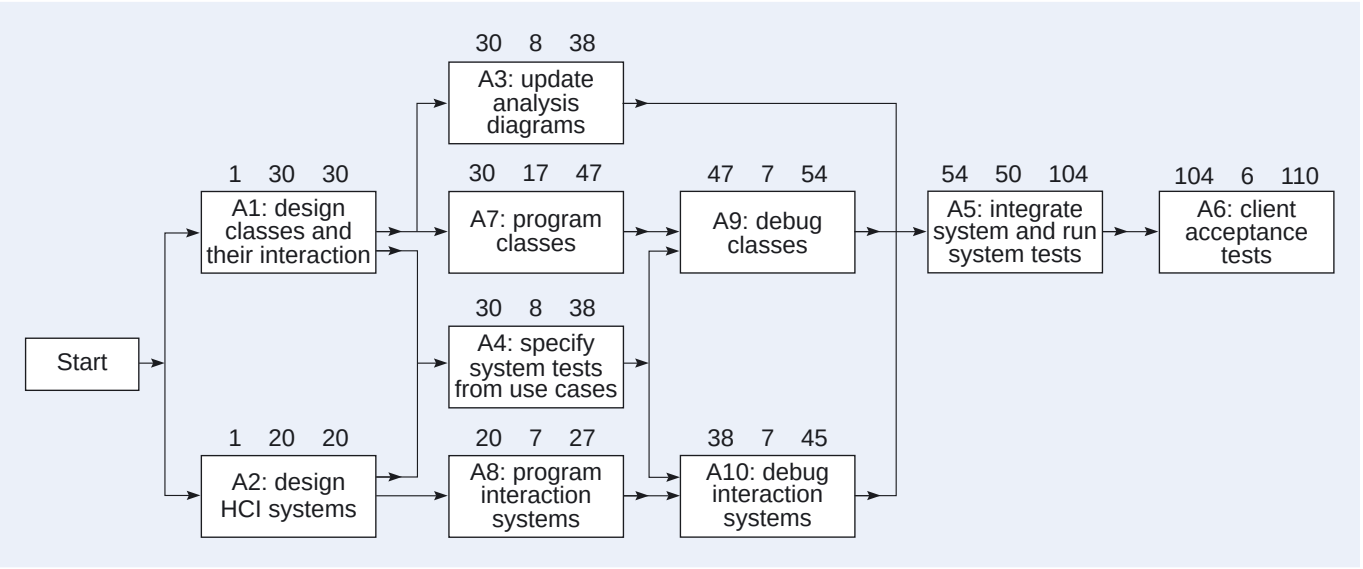

Figure 6 PERT chart for the project

You can see from the chart that there is some slack for some activities, such as A8, which does not need to be finished until just before the start of A10, and A10, which does not need to finish until just before the start of A5. However, other activities, namely those in the sequence A1, A7, A9, A5, A6, have no slack in them, and together constitute the **critical path** in the PERT chart. Therefore we cannot compress the timescale shown.

There is a lot more that can be said about PERT analysis, including how to deal with slippage, that we do not have the space to discuss here.

It is important to verify that the Gantt and PERT charts are consistent, that they end up with the same project duration. To perform this verification, we convert the number of working days in the PERT chart to calendar months so that we can compare the project length with the one in the Gantt chart. Recall that above, we converted person-months to person-days by multiplying by 18. The conversion we do now is the inverse: we convert working days to calendar months by dividing by 18. Thus we divide 110 (working days) by 18 (working days per calendar month), giving 6.1 months. Given the

accuracy in planning, the duration of 6.1 months from the PERT chart is close enough to the duration of 6 months from the Gantt chart. The durations are consistent.

The PERT chart in Figure 6 has several flaws. One is that (apparently) no time is allocated to fix bugs found during system testing. This problem shows one of the flaws of PERT charts; there are often many links between activities that the chart does not show. Another flaw is that the dependencies in the chart are simplified because implementation can almost always begin before design is finished. In fact, implementation often informs design, creating internal mini-cycles. The PERT chart in Figure 6 does not take this parallelism into consideration. Another problem is that the PERT chart in Figure 6 does not take resource conflict into consideration, which makes the activity start and stop times suspect.

Although PERT and Gantt charts are useful scheduling tools, the project manager needs to bear in mind that they cannot show everything.

SAQ 12

What is a deliverable and what is its importance for project scheduling?

ANSWER.....

A deliverable is an output, such as a design document or source code, of some software development activity of which the production gives evidence of the completion of the activity.

SAQ 13

What are the essential attributes of activities that are needed for project scheduling? How can project schedules be expressed?

ANSWER.....

The essential information required is the name of the activity, the effort required to complete the activity, and the dependencies upon other activities. The usual way to express the results of scheduling is either as a Gantt chart or as a PERT chart.

SAQ 14

What is meant by ‘resource balancing’?

ANSWER.....

This is the process of juggling the assignment of resources to activities over time so that each resource is used as near as possible to its maximum capacity without being overloaded, and each activity receives the resources that it needs.

3.5 Monitoring

In this subsection we shall consider the day-to-day process of *monitoring*, or *tracking*, the progress of a project, and replanning and reprioritising resources to meet new circumstances.

A large part of a project manager's time must be spent monitoring the progress of the project against the project plan (including the project schedule). Two ways of doing this are as follows.

- By collecting data on expenditure, especially of person-time, against activities, and thus knowing for each activity how much of the budget has been spent. This would typically be done at the end of each week. (On its own, this data does not tell you how complete the work is and the manager might also obtain some indication of this by collecting estimates of how much of each activity remains to be completed.)
- By monitoring actual completion of work through the acceptance of each activity's deliverable by some independent agent, such as a QA section or those who have to work to the deliverable.

Software engineers, and indeed people at large, can be hopelessly optimistic; so basing the monitoring of progress on expenditure and estimates of completeness can be very misleading. If the budget has run out, people often claim that they are '99% complete' week after week after week. Thus progress monitoring by deliverables and milestones can be much more reliable.

Almost every project falls behind schedule or goes over budget at some time in some of its activities. If these slippages are small, then these can often be made up by using the underspend and earlier delivery in other activities. But what do you do if the accumulated overspend and delay cannot be made up? Some spare resources might have been set aside for such contingencies, perhaps associated with each activity or set of activities. But this may not be a good idea, since people also have a propensity to use up any spare resources as if they had already been allocated them, a variation on the famous Parkinson's Law. If all else fails, the project manager has to plead for extra resources, or arrange to deliver less; or maybe completely re-estimate and replan the remainder of the project, using the increased knowledge currently available.

Parkinson's Law states that 'work expands to fill the time available'.

Timeboxing

One approach to planning and monitoring a project is to use *timeboxing*. Using this approach, the project is arranged to deliver something of use to the customer every one to three months. This could be the next version of a prototype within iterative development, or the next group of functions in an incremental delivery of the system, for example. The project is thus planned in broad terms as a sequence of *timeboxes*, but only the *next* timebox is planned in detail at any given time. Planning the details of software development over relatively short time periods means that there is less scope to go wrong. In addition, if the completion of the timebox is expressed in terms of a core set of functions that must be delivered, and a second set that may be delivered if time and effort permits, then the scope for missing deadlines is even smaller.

Timeboxing is usually associated with an approach called *rapid application development*, but can be used to great effect in any software development process. It is a form of the spiral development process mentioned in *Unit 1*.

SAQ 15

How does a project manager find out whether a project is on schedule or not?

ANSWER.....

A project manager collects data regularly (for example, each week) for each activity from which they calculate various statistics. These statistics could be on the effort spent, on the estimated effort to complete the activity or on whether the activity has been completed (as evidenced by its deliverables).

SAQ 16

What is the purpose of monitoring the progress of a project? Describe two ways of monitoring the progress of a project.

ANSWER.....

Monitoring is used to replan and reprioritise to meet new circumstances. First, on a weekly basis, say, collect data on expenditure against activities and compare it with the budget for the activity. Second, monitor the completion of work via handover of a deliverable to some other agent.

Tools

Planning and monitoring can be supported by tools. Planning work can be tedious, and tools help both to reduce errors and to reduce the tedium. This in turn means that the plans drawn up initially are more likely to be good, and, perhaps more importantly, that they are more likely to be kept up to date.

For estimation, historical data can be collected in a database of projects, which could be used for analogical estimation or to calibrate other estimation methods. The calculation of function points and the conversion of these to values for the effort and time required can also be supported by tools.

The scheduling of projects, particularly large projects, can benefit significantly from tools. These tools allow you to record work breakdown into a hierarchy of activities, to show the dependencies between these activities, and to record milestones. They allow a hierarchy of resources, from team to individual, to be made available, and then assigned to the activities. In addition, resource balancing can be supported through the automatic summation of efforts by individual and by activity, to make sure that all activities are adequately resourced and that people are not being under- or over-committed. Some tools also allow some automatic scheduling, but one must be careful of any 'equivalent heads' assumptions that the tool might make. When making a schedule for a software project, the particular skills of individual software engineers can be important, and the manager must be careful about assumptions that one engineer can be substituted for another.

Exercise 3

Suppose the library system that this section and *Unit 3* and *Unit 5* have used as an example is to be rebuilt completely, taking on much more sophisticated requirements, as follows.

1 Members are assigned a status defining what library services they have access to.

2 Some library items are held on reserve and cannot be borrowed, but members of suitable status can place a request to refer to them within the library.

3 Items can be photocopied by library staff, subject to copyright rules, and copies sent to members of appropriate status.

4 The library has a number of branches with different specialisations, and copies of items may be stored at a number of different branches.

5 Damaged items, or items that have not been used, may be withdrawn and given away to members.

Estimate the effort required for the development of this system.

Solution.....

The effort estimation requires that we know the use cases and classes involved.

The six original use cases (listed by actor) are:

- *Member*
  reserve book
- *Librarian*
  reserve book
  update catalogue
  issue copy of book
  return copy of book
  enrol new member
- *Browser*
  browse catalogue

To these must be added six new use cases:

- *Member*
  request reserved item
  request photocopy of item
  select withdrawn item
- *Librarian*
  register status of user
  photocopy item for user and send it to them
  withdraw item from library

The three original classes are *Book*, *Loan*, and *Member*.

To these we (arbitrarily) decide to add another another five classes corresponding to the requirements given in the question: *MemberStatusPrivileges*, *ReserveItem*, *PhotocopyItem*, *LibraryBranch*, *WithdrawItem*.

For this exercise we assume that the new use cases are not as complex as the original ones (which were weighted at 7), so we shall weight them at 5. So the FP contribution for the new use cases is $6 \times 5 = 30$, which added to the contribution of 42 for the original use cases gives an overall FP contribution for the use cases of $30 + 42 = 72$.

The new classes are just as simple as the original ones, so we shall weight them at 9 too, giving an FP contribution of $5 \times 9 = 45$ for the new classes and an overall FP contribution for the classes of $45 + 27 = 72$.

This then gives an FP value of $72 + 72 = 144$.

The complexity adjustment needs to be a little bit higher than previously because of the addition of branches and hence a distributed system. To account for this additional complexity we add an extra 5 to F_i, giving a value for F_i of $5 + 25 = 30$.

We now have:

$$FP_{\text{adjusted}} = 144 \times (0.65 + 0.01 \times 30) = 144 \times 0.95 = 136.8$$

Hence, using Table 1, this suggests approximately 8200 lines of object-oriented code, if we code in Java.

Using Basic COCOMO and assuming an exceptionally simple organic system (as in the earlier example), the parameters of Table 2 suggest that the effort required is about:

$$E = 2.4 \times 8.2^{1.05} \approx 21.9 \text{ person-months.}$$

Exercise 4

You are the project manager for a large complex software development project which breaks down into five subsystems of roughly equal size. You have estimated the size of the five subsystems at 100 KLOC each, but realise that if you developed them together certain common utilities could be identified, reducing the system size to 455 KLOC.

From the costs and timescale point of view, which should you do:

(a) develop the system as a single project in which the savings on the utilities could be achieved?

(b) develop the system as five independent subprojects in which the utilities would be duplicated?

Give reasons for your choice.

Solution

We would recommend (b).

To justify this choice, we shall use Basic COCOMO. Since the project is large and complex, we shall assume that the project and its subprojects are embedded.

For the single project of size 455 KLOC:

$E = 3.6 \times 455^{1.20} \approx 5571$ person-months (roughly 464 person-years);

$D = 2.5 \times 5571^{0.32} \approx 40$ months.

This implies a single team of about 140 software engineers.

For each subproject of size 100 KLOC:

$E = 3.6 \times 100^{1.20} \approx 904$ person-months (roughly 75 person-years);

$D = 2.5 \times 905^{0.32} \approx 22$ months

This implies five teams of about 40 software engineers each.

The overall effort required is about 464 person-years for a single project but considerably less, about 375 person-years, for the subprojects. However, the subprojects would have to be integrated, and this could be costly in terms of effort and time. Nevertheless, overall we might save by using the subproject approach. Furthermore, if the five subprojects were done in parallel, they would be delivered more quickly, even taking into account their final integration. Of course more people would be required, $5 \times 40 = 200$ rather than 140 for the single project approach, but that could be acceptable.

From this we conclude it is better to do the work as five subprojects.

3.6 Summary of section

Before starting a new software development, or at least before you have spent too much effort on it, you must be able to predict how much resource will be required. The resource required is mostly the time of software engineers and their managers. You saw how by drawing on experience you could assess the size of a prospective software system using function points, and from that predict the volume of software to be produced in lines of code, as a function of the particular programming language and development tools that will be used. You also saw how these size estimates could be translated into estimates of effort required, taking into account diseconomies of

scale. You also saw how to modify these estimates to take into account the complexity of the software to be developed.

A key point was that these were estimates, and that there must inevitably be some margin of error. Nevertheless, these estimates do form a reasonably sound basis for proceeding with a project and for developing a project plan. Once the project is running, the problem then becomes one of monitoring progress and ensuring that development proceeds according to the plan.

In particular, in this section you have seen:

1 that a project manager must analyse and control risk, and those risks can be categorised as project, technical and business risks;

2 that there are four strategies for dealing with risk: avoidance, retention, reduction and transfer;

3 that, in order to plan a project, the project manager must obtain estimates of the resources required by the project;

4 that there are two main factors that influence costs, namely system size and system complexity, that system size and system complexity are related, and that size is often measured in terms of thousands of lines of code, KLOC, in the delivered product;

5 four estimation techniques: estimation by analogy, estimation by work breakdown, function point analysis and COCOMO;

6 two ways of representing activities, their work content and their interdependencies, namely as PERT and Gantt charts, and their use in an example of project scheduling;

7 the relevance of monitoring the progress of a project for the purpose of replanning and reprioritising resources to meet new circumstances.

However, ensuring that costs are contained, and that the software is produced to the costs and timescales predicted, is not enough. We must also ensure that the quality is not compromised, and must set in place procedures for controlling quality and preventing compromises. This is the subject of the next section.

4 Quality management

In Section 2 you looked at process quality in general, and saw that there were a number of components to this. In Section 3 you looked at project management. In this section we move on to consider *quality management*, that is, how we assure the quality of the final software product. Quality management is an activity that takes place throughout the software development process. Quality cannot be added to a system at the end, it must be built in from the start.

We begin by motivating a concern for quality assurance by looking at how quality controls can be inserted into a project plan. Next, we shall consider the principles and components that constitute a software quality management system. Finally, we explore the ISO standards that govern the setting up of quality management systems in general, some software quality management systems in particular and an approach to setting up a software quality management system based on the notion of process maturity.

4.1 Adding quality to a project plan

In Section 2 you saw how to produce a work plan for a software development project. This included an estimate of the software engineering effort and other resources required, a list of the activities to be carried out, a schedule of when these activities would be carried out, and an allocation of appropriate resources to the activities. You saw that ideally every activity should produce one or more deliverables as evidence that the work had been completed.

To add *quality* to a software product, we need to identify when and how, in the development process, to apply product *quality controls*, that is, mechanisms for ensuring the quality of the deliverables of a software development process. For example, for the object-oriented systems developed in this course, we have been following a development process broadly in line with the Unified Process, using the notations of UML. Table 7 shows the quality controls needed for such a process in the case of the library example whose project plan we developed in Section 3. (Note that all deliverables are produced in conformance to some standard or guideline.)

Table 7 Quality controls for developing the software for the library system

Activity		Deliverables	Standards	Controls
A0	Analyse system requirements	Use case and class diagrams	UML	Reviews and inspections
A1	Design classes and their interaction	Interaction diagrams, fully specified classes, state diagrams	UML	Reviews and inspections
A2	Design HCI systems	Interaction diagrams, fully specified classes, state diagrams	UML and usability guidelines	Reviews and inspections

Activity A0 was not considered in Section 3, as it preceded the software development project considered there. We include it here for completeness.

Activity		Deliverables	Standards	Controls
A3	Update analysis diagrams	Use case and class diagrams	UML	Reviews and inspections
A4	Specify system tests from use cases	System test plan	Test guidelines and standard	Reviews and inspections
A5	Integrate system and run system tests	Tested system	Test guidelines and standard, usability guidelines	Reviews and inspections
A6	Client acceptance tests	Accepted system ready for operation	Test guidelines and standard, usability guidelines	Reviews and inspections
A7	Program classes	Code for server-side system	Java coding standard	Inspections and unit tests
A8	Program interaction systems	Code for client-side system	Java coding standard	Inspections and unit tests
A9	Debug classes	Code for server-side system	Java coding standard	Inspections and unit tests
A10	Debug interaction systems	Code for client-side system	Java coding standard	Inspections and unit tests

The quality controls in Table 7 are just those necessary for the technical work on the project; further quality controls will be necessary for the managerial work. These management quality controls are covered later in this section.

When we have specified all the quality controls for a project, we have developed a **quality plan** for the project. What we need to ensure is that all projects develop such quality plans, and then carry them out, avoiding the temptation to short-cut the quality controls if the project comes under resource or timescale pressure. It cannot be emphasised strongly enough that quality is not bolted on after the software has been built, it has to be built in from the very start.

SAQ 17

How are technical quality controls added to a project plan?

ANSWER.....

For each deliverable, we identify the quality control standards and guidelines to be applied, and the method of ensuring that they are applied.

4.2 Quality management systems

A **quality management system (QMS)** is an organisation-wide mechanism for building quality into projects and for managing the quality control process. Figure 7 illustrates the basic elements.

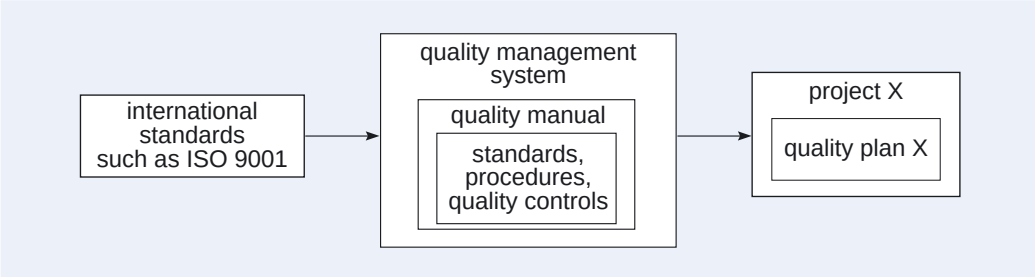

Figure 7 The architecture of a quality management system

A quality management system will include a **quality manual**, which in turn includes a number of standards, guidelines and procedures that are applied within the organisation. The quality manual will require that each project develops its own quality plan, which must comply with the guidelines laid down in the quality manual. The quality management system itself is likely to want to claim conformance to some external national or international standard for quality. ISO 9001 is such a standard.

- ISO 9001 is a *generic* standard, designed to be equally applicable to all organisations.
- ISO 9001 provides a generalised model for a QMS rather than a specification to be applied in an identical manner in every case.

ISO 9001 cannot be applied directly; to make a standard like ISO 9001 useful it must be implemented within an organisational structure. This is the task of the QMS of the organisation.

A QMS consists of a managerial structure, responsibilities, activities, capabilities and resources. An important part of a QMS is the *quality manual*. This document describes an organisation's QMS, and contains all the concrete details of an organisation's QMS (policy statements, standards, procedures, guidelines and so on). However, according to the guidelines in the ISO 9001 standard, the quality manual need contain only the policy statements which show, in general terms, how the model recommended by the standard is to be implemented in the organisation. The details of how this is to be done may also be contained in the quality manual, but they are more usually to be found in other documents.

Each project is required to prepare a *quality plan*. This is a document produced in accordance with the organisation's quality manual for a *specific* project run by the organisation. It addresses the quality factors that are important for *that* project and, in addition to the standards and procedures that are applicable to that project, lists the quality controls to be used to ensure the achievement of quality factors. The quality plan acts first as a contract between the client and the developers, detailing the quality factors that apply to that project; and second as a contract between the development team and those people who are to carry out the quality assurance function. The idea that a quality plan forms a formal contract is very important; it provides the customer with an opportunity to approve the approach to be taken by the developers, including any mechanisms for making changes to requirements if they become necessary. A quality plan is a mechanism for 'focusing' the QMS onto the project.

SAQ 18

(a) What is a quality management system (QMS)?

(b) What would you expect to find in a QMS?

ANSWER.....

(a) A QMS is a mechanism adopted throughout an organisation for building quality into its projects and for managing the quality control process.

(b) A quality manual that describes an organisation's QMS, including managerial structure, responsibilities, activities, capabilities and resources. One sometimes includes the quality plans for projects in the quality manual, to help learn from history.

4.3 The ISO 9000 family of standards

We have looked at the elements of a quality management system and mentioned the standard ISO 9001. We now look at ISO 9001 in more detail, and at how a software company's QMS can be certified as conforming to this standard.

ISO 9001

ISO 9001 is one member of a family of international standards, known collectively as ISO 9000 and published by the International Standards Organisation (ISO).

There are collections of guidance documents to support the standards. Those in the ISO 9000-x series, such as ISO 9000-3, provide guidance on the application of the standard; those with higher numbers, such as those in the ISO 9004-x series, provide guidance on specific aspects of quality management systems.

ISO 9001 is not industry-specific and describes the quality system used to support the development of any product that requires design. It applies to all steps from design right through the manufacturing process.

The fact that ISO 9001 is not industry-specific does raise a problem; it is expressed in general terms and needs a high degree of interpretation by developers of different products. ISO 9000-3 (discussed later) addresses this problem, in that it offers guidelines on how to interpret ISO 9001 in the case of software development.

Accreditation and certification

If someone says they have implemented ISO 9001, how can we judge whether the implementation is in the spirit intended by the standard? This problem is addressed by *certification* or *registration*. Within this mechanism, a competent body, known as a certification body, makes an assessment of an organisation's implementation of ISO 9001 and judges whether the implementation meets the requirements laid down by the standard; if so, the organisation is placed on a *certification register* of companies certified as meeting the requirements of the standard. Anyone wishing to know if a company is certified can find out by consulting the register.

This, however, creates another problem: how can a company wishing to achieve certification tell whether a certification body is competent to perform an assessment? This problem is addressed by another mechanism, known as *accreditation*. While any certification body may offer to conduct an assessment and issue a certificate of compliance, only those certification bodies that are accredited may offer an accredited certificate of compliance, and it is the accredited certificate in which the industry places its trust. Furthermore, accreditation is offered only in those areas in which the

certification body can demonstrate that it is competent to perform certification audits. Accreditation is 'scoped' to indicate the industry sectors within which a given certification body is deemed to be competent.

Accreditation is itself subject to a mechanism similar to certification. Certification bodies must comply with a standard; this lays down standards for the management system employed by the certification body and standards of competence for its *assessors* (also known as *auditors*). The competence of auditors is defined by another organisation, the International Register of Certified Auditors (IRCA), which requires certain levels of training and experience in an auditor before they can be placed on the register. Certification bodies use only registered auditors to conduct certification audits within their scope of accreditation.

In the United Kingdom, the *TickIT scheme* has been introduced to deal with quality certification in the software industry. TickIT is an accreditation scheme to guarantee that registered auditors are competent and experienced in software development, and this increases the confidence of companies applying for certification.

ISO 9000-3

The specific needs of the software development process have been recognised by the ISO, and a set of guidelines for interpreting ISO 9001 in this context have been published as ISO 9000-3. A list of section and subsection headings for ISO 9000-3 is shown in Table 8.

Table 8 ISO 9000-3 clauses

4. Quality system: framework	6. Quality system: supporting activities
4.1 Management responsibility	6.1 Configuration management
4.2 Quality management system	6.2 Document control
4.3 Internal quality system audits	6.3 Quality records
4.4 Corrective action	6.4 Measurement
5. Quality system: life-cycle activities	6.5 Rules, practices and conventions
5.1 General	6.6 Tools and techniques
5.2 Contract review	6.7 Purchasing
5.3 Purchaser's requirements specification	6.8 Included software product
5.4 Development planning	6.9 Training
5.5 Quality planning	
5.6 Design and implementation	
5.7 Testing and validation	
5.8 Acceptance	
5.9 Replication, delivery and installation	
5.10 Maintenance	

We shall now look at each of the sections of ISO 9000-3 in more detail.

Quality system: framework

ISO 9001 and ISO 9000-3 both envisage four components of a quality management system.

There must be *management responsibility*, in that the senior management of an organisation must define its quality policy and formally commit the organisation to provide the resources necessary to enable that policy to be implemented effectively. The quality policy is promulgated through the quality manual.

The *quality management system* should be documented as a quality manual incorporating a set of procedures, standards and other documents, so that the system is defined and can be audited.

This documentation is mandatory if the system is to conform to the requirements of the ISO standard.

There must be *internal quality audits*, through which the effectiveness of the QMS implementation can be judged and improvements identified. These audits must be planned so that all aspects of the system are audited on a regular basis by staff trained for the task. The purpose of the audit is to identify any areas of non-conformance with the ISO standard and to reveal and correct any system weaknesses. This internal audit complements the external audit carried out by the certification bodies.

There must be *corrective action mechanisms*, through which problems and potential problems can be identified and corrected or avoided. The primary purpose of such mechanisms is to ensure that all errors in documents or software are formally reported, tracked and resolved. The performance of this part of the system is therefore critical to the overall performance of the QMS. Implicit in these mechanisms is an *improvement mechanism*, by means of which the QMS is regularly analysed to ensure that it continues to be effective and appropriate. The data from internal audits, corrective actions and product and process metrics can be used by senior management to review performance and identify opportunities for improvement.

A *process metric* is a measurement of cost, effort, productivity or time, or any other measurement associated with the process.

These four components are built into ISO 9001 (and mirrored in ISO 9000-3) to ensure that an effective QMS is implemented and maintained. We could go much further; once the basic monitoring and decision making is established, these components can be used to identify and implement positive improvements as well as to avoid problems.

Quality system: life-cycle activities

The principal guidance in ISO 9000-3 on life-cycle activities is that a life-cycle model should be used as the basis of a QMS, and that all quality activities should be clearly related to the life cycle. No particular life-cycle model is preferred.

You will see from the list of clauses for ISO 9000-3 in Table 8 that each phase of the life cycle is represented, but the requirements analysis phase gets particular attention, and the guidelines include advice to purchasers as well as to suppliers. The only other area receiving the same level of attention is planning.

Quality system: supporting activities

The supporting activities in ISO 9000-3 include configuration management, which is less prominent in the ISO 9001 format, and measurement, which is almost buried under the heading of ‘Statistical techniques’ in ISO 9001.

SAQ 19

What is the difference between *certification* and *accreditation* in connection with ISO 9001?

ANSWER.....

A company must be *certified* as conforming to the standard by a body that is *accredited* to carry out the certification.

SAQ 20

(a) Who is formally responsible for the quality of an organisation's delivered products (or services), and how is this responsibility discharged within a QMS based on ISO 9001?

(b) How does a quality audit relate to a quality plan and a quality management system?

ANSWER.....

(a) The senior management, who make their quality policy known through the quality manual and authorise the development of a QMS designed to implement that policy.

(b) The reason for quality audits and their frequency should be specified in the quality plan for a project, in order to conform with the requirements of the quality management system.

Process maturity and process improvement

One of the problems with ISO 9000 is that it really only demands that organisations work consistently, without any notion of relative competence. The intention had been to require that best industrial practice is followed, but how that was to be determined was not clear. But an important aspect of ISO 9000 has been **process improvement**: that an organisation should learn from difficulties encountered with their existing processes, and improve as a result. This aspect led to the idea of the levels of organisational competence through which an organisation should progress.

Capability maturity model integration (CMMI)

More information about the CMMI can be found at http://www.sei.cmu.edu/cmmi (Accessed February 2008).

The maturity levels are described on pp 35–40 of the document CMMI for Development, Version 1.2, found at http://www.sei.cmu.edu/pub/documents/06.reports/pdf/06tr008.pdf (Accessed February 2008).

The Software Engineering Institute at Carnegie-Mellon University in the United States has produced a model to categorise the maturity of the software development processes of an organisation. The authors of this model believed that one of the reasons for problems in software development lies in the inability of an organisation to manage its software development processes. One key to managing anything is to understand what it is you are trying to manage, and the *capability maturity model integration* (CMMI) can be used as a measure of an organisation's corporate understanding of its development processes.

A *software process* is a set of activities, methods, practices and transformations used to develop and maintain software and associated products, such as project plans, design documents, and so on. The CMMI identifies five levels of software process maturity, as shown in Table 9.

Table 9 The five CMMI levels of software process maturity

Level	Description
1 Initial	The software development process is largely undocumented and success depends on individual effort and some good fortune.
2 Managed	Basic project management controls are in place to manage costs and development schedules. Previous successful practices are repeated.
3 Defined	The processes for development and project management are documented and used by all projects, but may not be clearly understood by either managers or developers.
4 Quantitatively Managed	The development process is monitored as regards process and quality. The process is well understood by both managers and developers alike.
5 Optimising	The development process is continually improved on the basis of feedback from the monitoring process.

It is generally accepted that an organisation that is capable of gaining ISO 9000 certification is necessarily at CMMI level 3 — it has shown that it can define its processes, and as a result of that its processes will be 'managed' in the CMMI sense. However, it may well not be 'quantitively managed' in the CMMI sense, let alone 'optimising'.

SPICE

In August 1998 another ISO standard was released: ISO 15504, also known as SPICE (Software Process Improvement and Capability dEtermination). The standard was revised in 2004. The objective of ISO 15504 was to blend ISO 9000 and the CMMI. The overall concept is illustrated in Figure 8.

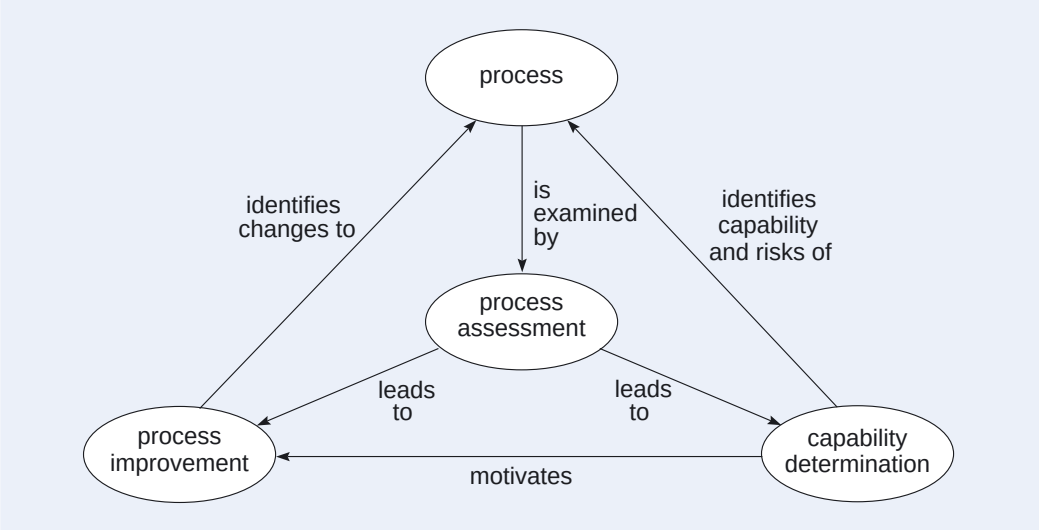

Figure 8 ISO 15504 software process assessment model

Any software development organisation has a software development process that can be assessed to determine the underlying capability of the process. This assessment may suggest shortcomings of the process, which then motivates a process

improvement strategy, which is then used to change the process. This continues in cycles as needed.

SAQ 21

How could an organisation continuously improve its software development processes and move up CMMI levels?

ANSWER.....

The organisation could document the way work is performed, train its staff and gather and analyse historical data about its performance. In new projects, the organisation could measure performance with the aim of improving it.

Exercise 5

What sorts of thing could be measured during internal quality audits and corrective actions to help to improve the effectiveness of a quality management system?

Solution.....

- Internal quality audits: the number of audit non-conformances, where they were found and which clauses of the standard were infringed.
- Corrective actions: the number, severity and type of errors; the source of errors (which phase of the software development process) and the location of errors (which department or project).

Exercise 6

Using software project planning as an example, discuss how the basic techniques of the CMMI might evolve as an organisation goes from maturity level 2 to maturity level 5.

Solution.....

- At level 2, project level schedules are appropriate.
- At level 3, a common approach to project planning is required across all projects.
- At level 4, it is appropriate to construct a common estimation method with feedback of actual results to measure estimation effectiveness.
- At level 5, improved estimation techniques and tools should be sought.

Exercise 7

Can you see any obvious similarities and differences between the ISO 9001 approach and the CMMI?

Solution.....

The CMMI has a wider scope than ISO 9001, so there are key process areas that have no equivalent ISO 9001 heading. Where the two approaches address the same areas, they are broadly equivalent.

4.4 Summary of section

In Section 3 you saw how the work on a project can be planned; In this section you saw how quality controls could be added to this work plan.

Usually quality controls are regulated at an organisational level, through a quality management system that would conform to ISO 9001 or equivalent. You saw how this happens through the QMS and its quality manual requiring that each project produces and then conforms to a quality plan.

You also saw how to compare quality management systems between organisations through the CMMI and ISO 15504.

5 Configuration management

A typical software development project will produce thousands of separate items that are interdependent. These may be documents; UML or other design models; code artefacts such as Java code, compilation specifications, and libraries; and test specifications, unit and system test code, and regression test specifications and code. All of these are incredibly easy to change, and during the life of a project many versions of each item will be produced. Unfortunately, not all versions of the items will work together properly, and we need to be very careful to keep track of what does work together. This is the task of configuration management. Configuration management is an extremely important task in large projects, software or otherwise, but it is often overlooked. Even small software projects should use a version control system (part of a configuration management system), but many don't.

A software version control system keeps track of changes to individual files, helping determine what was changed, when, and why. Very importantly, it helps with backing out changes that break the system. Examples of open source version control systems include RCS, CVS and Subversion.

As soon as any item has been produced on a project, it may be used as the basis for further work. If it needs to be changed, the changes may need to be propagated throughout the system, possibly leading to changes to all items derived from the changed item. Change needs to be controlled, and *change control* is a central part of configuration management.

Configuration management and change management are the subject of this section.

5.1 Configuration items

During the development of software, a variety of items are produced, such as:

- use case diagrams, and other UML diagrams;
- individual Java programs or classes, or the compiled versions of these;
- architectures and patterns;
- test plans and test cases.

For each type of item, there may be a large number of different individual items produced. There will be many use case diagrams, many interaction diagrams, and so on, which are produced during the project, stored, retrieved, changed, stored again, and so on. We shall refer to these items as **configuration items**.
Each configuration item must have a unique name, and a description or specification that distinguishes it from other items of the same type.

When a configuration item is changed, or *revised*, a new **version** (or *revision*) of the configuration item is produced. It is the versions of a configuration item that are stored and retrieved. We often talk informally of an item when we mean a version of an item, just as we often talk informally of a class when we mean an instance of a class. To avoid misunderstanding, here we shall not abbreviate 'version of an item' to 'item'. Versions of items are given *version numbers*, which are unique with respect to that configuration item.

We must also distinguish between **variants** of a configuration item; these are essentially the same item but differ in some well defined sense. For example, the

configuration item 'library class diagram' might be available in two variants, one for centralised libraries and one for distributed libraries with branches. Variants are distinguished by having different, descriptive names, and also have descriptions characterising them. Variants themselves may undergo revision and thus have versions.

As the software is produced, one item leads to another. A use case diagram may be analysed to produce a class diagram, a class diagram and a use case diagram may lead to an interaction diagram. This sequence is important information, since if we update a version of one item, all the versions of other items that were based on this item probably need to be updated as well to keep in step. We call these versions of items based on other items **derivations**.

Figure 9 gives a class diagram for configuration items, their variants and the versions of both.

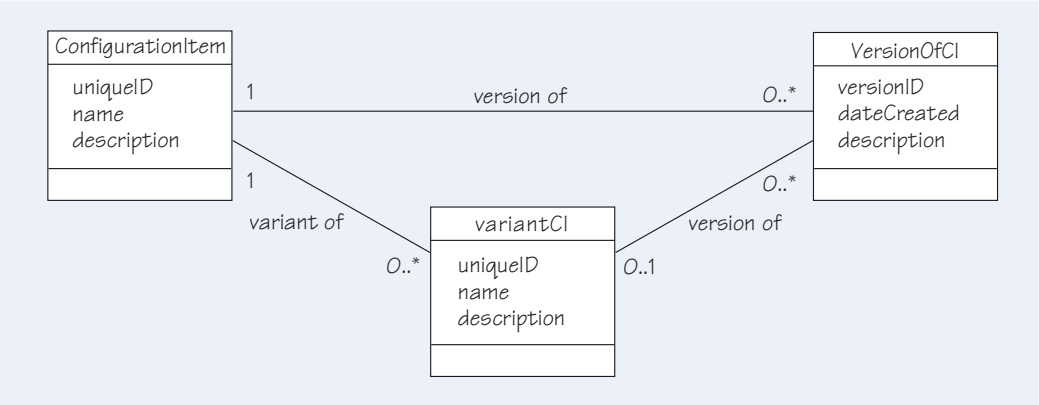

Figure 9 Class diagram for configuration items

Configuration items would normally be stored in machine-readable form in a project *repository* or *library* or *database*, which we shall refer to as the **configuration repository** (or just **repository**). Placing a version of an item into the repository is usually termed **checking in**, whilst retrieving a version of an item from the repository is usually termed **checking out**.

When an item version is checked out, the current **preferred version** (usually the latest) is obtained unless a specific version number is indicated. The time of check out and the person who did this is recorded. Several people may check out the same item simultaneously — many might only want to refer to the item, but some may also have the intention of updating it. If the item is updated, then at check in, a new version could be created, and the originator of this version recorded together with the time it was created. This may seem all right, but what if two people check out the same version of an item, then both update it and later both check it back in? Here are ways we might control the situation where two people have changed the same version of an item and then attempt to check it in as a new version.

1 Accept only the first one checked in, and disallow any later check ins. Only the new version is then allowed to be updated, after it has been checked out.

2 Elaborating on point 1, notify other users of a particular version as soon as the first new version of that older version is checked in, to warn them should they be planning an update.

3 Allow later check in actions to create different versions even if they were checked out from the same earlier version; usually this is done using a branching structure of derivations.

4 Allow two (or more) parallel development streams, as in point 3, to be merged later, either manually or using tools.

It is the last option that is usually chosen, and so we shall assume that that option has been chosen.

The various actions can be documented as a use case diagram, as in Figure 10. Only the manager can create new configuration items and variants of them, and set the preferred version on which all developers will work. Developers check out item versions, usually the preferred version though maybe occasionally a different one. When they check an item version back in after changing it, all check ins after the first produce a branched line of versions which can be later merged.

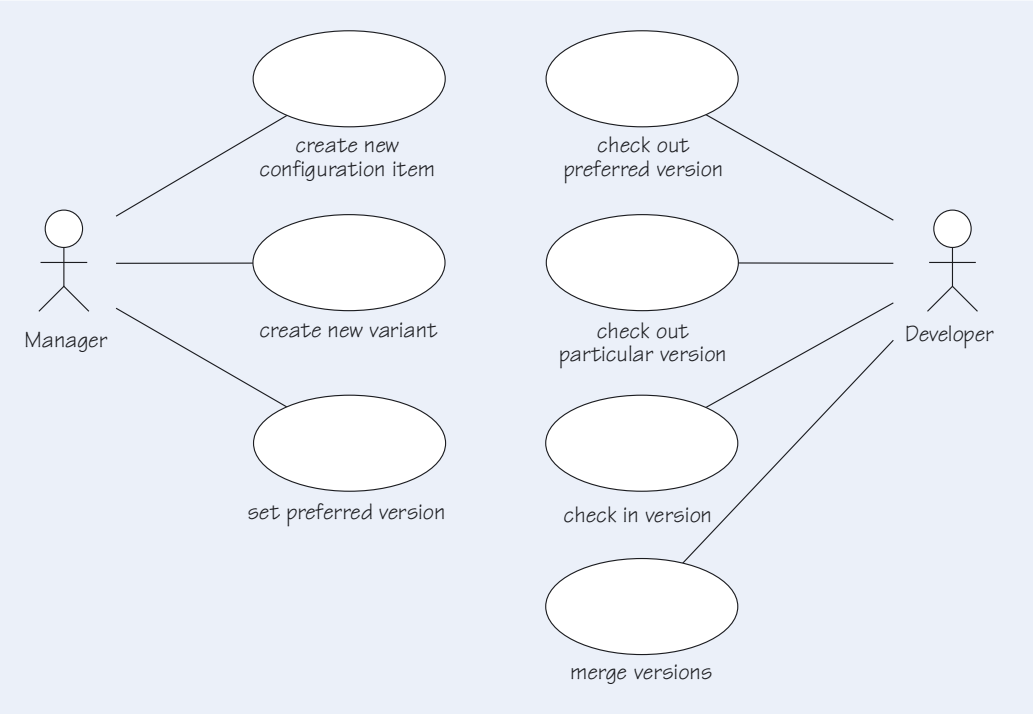

Figure 10 The use case diagram for the documentation of the configuration management process

SAQ 22

Distinguish between the terms *configuration item*, *version*, and *variant*.

ANSWER.....

A configuration item is any elementary work product produced during a project. A configuration item will exist in many versions, which are revisions of the original item. It may also possess a number of variants, which are essentially the same as the original item except that they differ in some well defined part of their description; variants may themselves exist in many versions.

SAQ 23

What should happen if two people check out the same version of an item, both people update it, and then attempt to check their updated version back in again?

ANSWER.....

A frequently used project policy is that both check in operations are successful, creating new versions recorded as derived from the same original version, but now on independent paths of development. These could be merged later by invoking the appropriate operation.

Configurations and baselines

As we develop a number of configuration items, with their sequences of versions, we find that particular versions of items belong together. For example, we might have developed version three of the sequence diagram based on version two of the use case diagram and version seven of the class diagram. We want to mark selected versions of each item as belonging together to make a consistent whole.

We call the collection of items necessary to construct a product, a **configuration**. When an instance of a product is constructed, particular versions of the items in the configuration will be used, and this set of versions is called a **configuration version**. Some configuration versions are singled out as special because they form a foundation from which further development can progress. These are called **baselines** and are typically associated with the major milestones of a project. We talk of further development progressing from a baseline. The configuration versions delivered on completion of the project for use by the customer are called **releases** and are the baselines from which maintenance takes place.

A configuration can be thought of as configuration item, albeit consisting of a collection of other configuration items. Thus we can think of configurations as containing other configurations, which in turn can contain other configurations, and so on.

SAQ 24

Explain what a *release* is, in terms of configurations.

ANSWER.....

A release is a baseline created at the completion of a stage of project, and includes everything that is delivered at the end of that stage, including executable files, documentation, and possibly regression test cases.

5.2 Change control

So far we have focused on configuration items and the versions of these that arise through change. Our primitive actions of checking in and checking out go some way towards regulating change, but we really need something more formal, otherwise anybody could make a request for a new function for the system, and a willing developer could well respond by making the change uncoordinated with other changes taking place. We need a way to manage this situation.

Baselines give us one instrument, and we talk of **freezing** a baseline, forbidding any further change to it. That means disabling further check in operations for the items in the corresponding configuration. But that is not always appropriate, and we need to have some means of updating a baseline in a controlled manner. This is what **change management** achieves.

One change management method requires that before a change can be made, a formal request for a change to the baseline must be made. The person requesting the change may be a user, or a software engineer working on the project or somebody else; they might want some new facility or have discovered what they believe to be a defect. The **change request** makes the case, with as much supporting evidence as possible.

On receipt of a change request, further information might be added: for example what needs to be changed (the main item concerned, and all the other things that must be changed in consequence, discovered through an impact analysis) and an estimate of the cost and timescale. This is then put to a designated authority: this might be the project manager on a small project, or a change control board (CCB) for a large project. A decision to proceed with the change, or to defer or reject it, is made.

Once a change has been agreed, the change is undertaken when resources are available. Several changes to the same items may be grouped together, and some priority system may operate. To make the change, the work goes through a small-scale development process:

- check out the versions of the configuration items required;
- design the changes, updating the UML diagrams as appropriate;
- make the changes to the code;
- unit test the changes systematically;
- check changed versions back into a new configuration;
- regression test the changes systematically;
- gain formal approval for the changes;
- make the new configuration into a new baseline or release and inform parties affected by the change.

The process we have been describing will not cater for all circumstances. Sometimes there is a need for emergency repairs, which must be treated as a special case. Here some person in authority must still agree the change, with it being cleared with the CCB or whomsoever in due course. Versions of configuration items are checked out, but changes may now be made immediately to the code, which is then tested and immediately put into service. UML diagrams and formal incorporation into a new baseline come later, but must still be done. Undertaking changes in this way must only be done in exceptional circumstances.

Figure 11 shows a possible change control procedure.

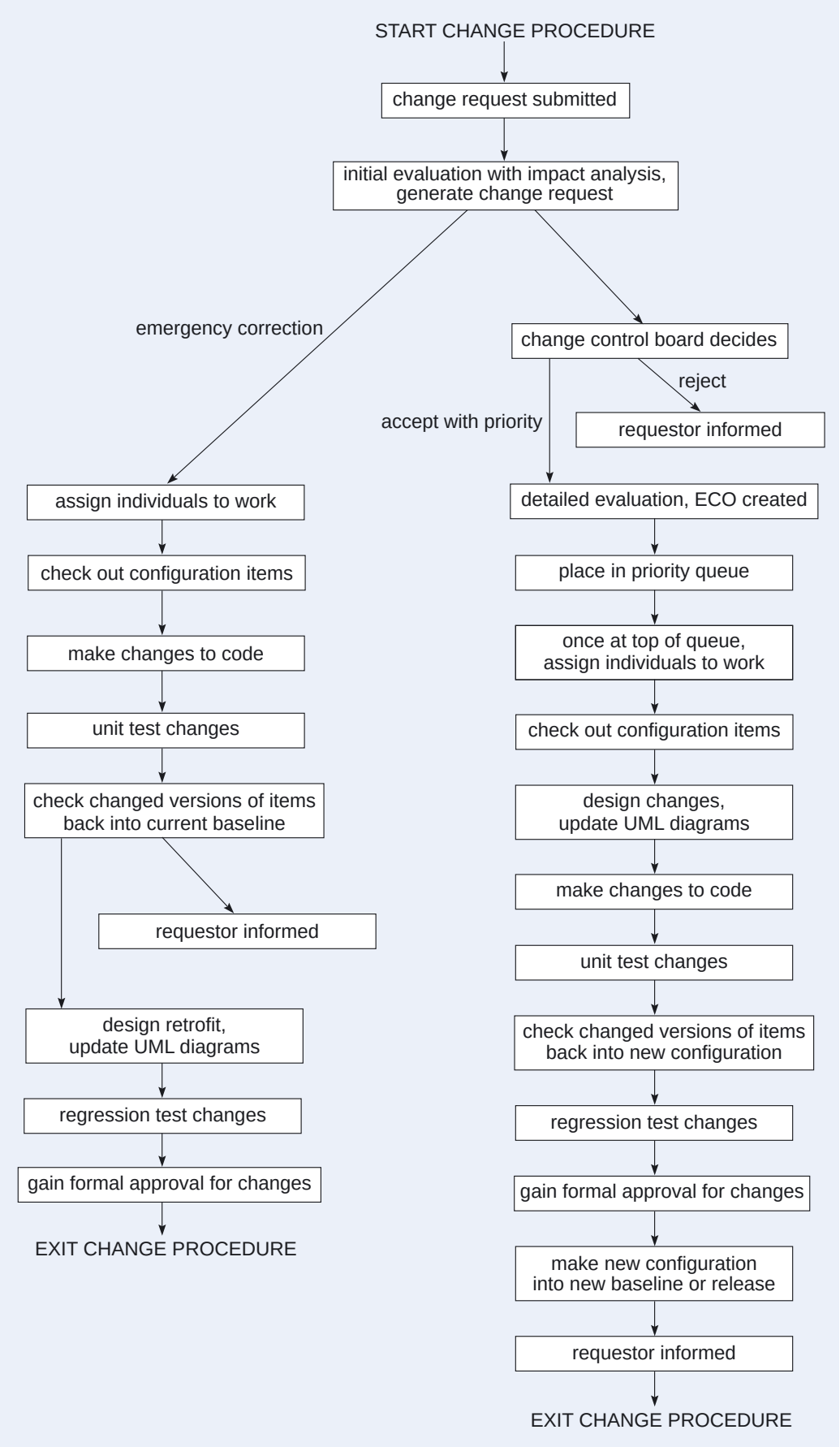

Figure 11 Change control procedure

Impact analysis is the process of finding out the likely effect of making the change, to estimate the amount of change required.

ECO = engineering change order. Some texts use ECO to mean emergency change order, which is different.

Here retrofit means to revise the UML diagrams to incorporate the revised unit. Normally, code would be written to conform to an already designed UML model. Here we are changing the code first and then amending the UML model to suit.

This change control procedure, and the underlying configuration management system described earlier, form an important part of the organisation's quality management system, and should be described in the quality manual. An individual project should declare, in its quality plan, when the baselines are produced, and who serves on the change control board. Special circumstances in the project may simplify the general system described here; for example, variants may not be important, and changes may not be given priorities.

SAQ 25

How would a change control board influence an emergency change? Why would it want to?

ANSWER.....

Since an emergency change is done before being cleared with the CCB, the only action that the CCB might take is to retrospectively request the change be undone.

All changes, even emergency changes, can have negative consequences. In some circumstances it may be better to live with the problems identified than with the adverse effects introduced by making the change.

5.3 Tools

Configuration management needs to be supported by tools. Based on the descriptions above, it could be possible to build a suitable tool ourselves, although it is not recommended except perhaps if the system is very small. We would need a database management system for our repository, but, since there are only likely to be a few thousand configuration items in a small system, with only tens or at most a few hundred versions of each, a simple database management system would suffice. Software to compute differences between versions of text files would also be required.

There are a number of very good configuration management systems available commercially. For example, the Rational Division of IBM offers the ClearCase system, which at the time of this writing is a market leader. But there are many others that are very good too. Open source products like RCS, CVS, and Subversion are good version control systems, and can suffice as configuration management systems in many cases. The most sophisticated configuration management systems are able to handle repositories of hundreds of thousands of configuration items, and millions of versions, with high security, possibly including the encryption of the items in storage. They are also able to support distributed software engineering teams, and inform users of any changes to versions of configuration items that are currently being worked on or used.

It is worth noting that *all* changes benefit from the support of version control. Consider the writing and running of unit tests and then fixing bugs. This activity generally won't involve any changes to specifications or designs, so no change requests are involved, but it is still essential to keep track of the code versions. We should consider using a repository, even for quite small projects.

Exercise 8

Consider the library system used as an example in this unit. Identify the configuration items, versions, variants, configurations and baselines in this example.

Solution

Configuration items:

- UML deliverables: the use case, class, interaction, and state diagrams;
- the code for each class;
- the unit tests for each class;
- the test plan, covering integration, acceptance and system tests.

Versions: for each of the configuration items.

Variants: no variants have been specified, though in general one could imagine variants for different kinds of library, from pure reference libraries to libraries containing a wide range of items including DVDs, CDs, audio books, etc., to libraries with branches and mobile vans.

Configurations: any consistent set of versions.

Baselines: these would be configurations that would be reviewed and then frozen at key milestone points, such as:

- after the completion of design, near the end of month 3 in the Gantt chart in Table 6;
- after system integration, testing and debugging, near the end of month 6, but before the acceptance tests;
- after successful acceptance tests, as the first release.

5.4 Summary of section

In this section you saw that the development of software involves the production of many, perhaps many thousands, of configuration items. Each configuration item undergoes a sequence of revisions as it is developed and changed, to produce new versions of items, possibly on parallel threads of development that might be later merged. Not all versions of items will necessarily work together, and consistent sets of versions of items are called configuration versions. The important configurations are the baselines established at important points in the development process, and releases that are delivered to customers.

Changes to versions of items, particularly when they form part of a baseline or release, cannot be made lightly because of the knock-on consequences of making any change. Thus, changes will usually be closely managed with a well defined procedure, perhaps involving a change control board.

Configuration management forms an important part of an organisation's quality management system.

6 Summary

This unit went beyond the technical aspects of software development and explored how the software development process can be made predictable in terms of costs, timescale and quality of product. You saw the principles underlying the management of software development, and how this breaks down into the management of resources, quality and configuration.

You also saw how resources are managed (by means of project organisation, estimation of work content and cost, scheduling of work and progress control) and how software quality can be assured (by marshalling the product quality measures in *Unit 11* and by drawing upon external standards such as ISO 9000-3 and CMMI).

This unit also showed you how the software product's configuration can be managed.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- describe the importance of building quality into the software development process;
- discuss how process quality can be improved through better management of projects, people, quality and configuration;
- describe how project management is concerned with controlling costs and ensuring that software is produced on time, whereas people management is concerned with evolving methods of enabling people to work successfully together;
- discuss the importance of analysing and managing risk;
- estimate the resources required by a project;
- represent activities, work content and interdependencies using PERT and Gantt charts;
- describe quality management system (QMS) and ISO standards;
- describe how to improve the software development process with reference to CMMI and ISO 15504;
- discuss the importance of a change control procedure.

References

Boehm, B. (1981) *Software Engineering Economics*, Prentice Hall.

Brooks, F.P. (1975) *The Mythical Man-Month*, Addison-Wesley.

Fetcke, T., Abran, A. and Nguyen, H. (1998) 'Mapping the OO-Jacobson approach into function point analysis', in *Proceedings of TOOLS USA 97: Technology of Object-Oriented Languages and Systems*, Santa Barbara, California: 1997, IEEE.

Pressman, R.S. and adapted by Ince, D. (2000) *Software Engineering: A Practitioner's Approach*, 5th edn (European adaption), Maidenhead, McGraw-Hill.

Index